

An Efficient Tree-Structure Hash: QuantaSylva Hash (QSH)

Submission to NGCC ICCS

Authors

Shiqi Hou, Tsinghua University

Dongdai Lin, Institute of Information Engineering, CAS

Shichang Wang^(✉), Nanyang Technological University

Contact

Shichang Wang, Personal Email archie_wong@outlook.com,
Primary Email shichang.wang@ntu.edu.sg

Tuesday, June 30, 2026

Contents

1	Introduction	4
2	Parameters and Notations	5
2.1	Recommended Parameters	5
2.2	Notations and Operations	5
2.3	Domain-separation Flags	6
3	The CHACHA BAHRU Permutation	7
3.1	Transformation function G on Word-Quadruple	7
3.2	Linear Layer : Shuffle P_{16}	9
3.3	Round Function $R_{d,w}$	9
3.4	Permutation $E_{d,w}$ and CHACHA BAHRU	11
4	Compression Function via Modular Addition	13
5	Specifications of QSH Algorithms	14
5.1	Padding Message	14
5.2	Parsing Padded Message into Chunks	15
5.3	Computing Chunk Hash Value	15
5.4	Generating Message Digest	16
6	Design Rationale and Features	17
6.1	Principles of CHACHA BAHRU	18
6.2	Features	19
7	Security Claim and Analysis	21
7.1	Security Claim	21
7.2	Analysis of the Construction	21
7.2.1	Tree-based construction	21
7.2.2	Compression function with modular addition	22
7.3	Analysis of the Permutation—CHACHA BAHRU	22
7.3.1	Differential-trail bound property	22
7.3.2	Linear-trail bound property	25
7.3.3	Rotational and Rotational-XOR property	26
7.4	Analysis of the Algorithm QSH	28
7.4.1	Preimage attacks	28
7.4.2	Collision attacks	29
7.4.3	Analysis in quantum setting	29
7.5	Analysis of Specific Attacks	30
8	Performance of QSH	31
8.1	Software Implementation	31
	Appendix	40

A	Algorithm 1 when $d = 3$	40
B	Proof of Proposition 1	40
C	Lower Bound on Active G	43

1 Introduction

This document specifies a family of hash function algorithms QUANTASYLVA HASH (abbreviated as QSH throughout the following parts), which provides three variants: QSH-512, QSH-768 and QSH-1024. All of them are built from a single compression function, CHACHA BAHRU, and differ only in their internal parameters and output length.

The QSH family and especially the underlying permutation CHACHA BAHRU have been designed for robust security and high throughput in software while keeping the description and the implementation simple. The compression function is defined entirely in terms of ARX operations—modular addition, bitwise rotation, and exclusive-OR on 64 words—and is organized in a tree-based mode that exposes instruction-level parallelism. The tree-based mode is a central point of QSH’s software performance. Unlike serial modes such as Merkle–Damgård [40, 21] or Sponge constructions [13], in which the compression function must be evaluated sequentially over successive message blocks, the tree mode allows independent subtrees to be processed in parallel. On long messages, this turns what would otherwise be a strictly serial computation into a partially parallel one, and the resulting parallelism is readily exploited by the multiple execution units and wide SIMD registers of modern processors.

In addition, the permutation lends itself well to fast vectorized implementations on 64-bit platforms with wide SIMD registers, while bit-interleaving allows for straightforward implementations on 32-, 16-, and 8-bit platforms. The absence of S-boxes and lookup tables makes constant-time implementation immediate on all targeted platforms. To be short, QSH achieves particular efficiency in high-throughput scenarios such as hashing of large files, bulk data authentication, and Merkle-tree-based applications. For example, on x86-64 AVX2 platform, QSH-512 runs at approximately 8.52 cycles per byte (328 MB/s) in one thread, and up to 1855 MB/s in 12 threads for a single message.

The design of QSH is built on two contributions that we believe are of independent interest. Firstly, we propose a new message absorption mechanism for the construction of building a compression function from a large permutation: a simple Even–Mansour-like (JH-style), keying combined with message-dependent modular addition. The construction is lightweight and admits a high algebraic degree after a small number of rounds, as a robust line of defense against algebraic attacks. Secondly, we propose a generalized ARX design methodology that extends the classical ARX paradigm to multi-dimensional state arrays: a simple and efficient quadruple-word transformation is applied iteratively to the array, followed by a shuffle that permutes its entries. The two-dimensional instance of this methodology recovers ChaCha; CHACHA BAHRU is the three-dimensional instance and serves as the underlying permutation of QSH.

We have selected the round numbers and parameter sets for QSH-512, QSH-768, and QSH-1024 with a comfortable security margin against all attacks known to us. The margin rests on two independent design choices. The new JH-style compression-function construction with double-message-injection by modular addition—admits a high algebraic degree after only a few rounds, which we

regard as a robust line of defense against algebraic and structural attacks on the mode. The underlying ARX permutation CHACHA BAHRU, in turn, inherits the well-studied diffusion and nonlinearity behavior of the ChaCha family and extends it to a three-dimensional state, providing ample resistance against differential, linear, and rotational cryptanalysis. The two layers reinforce each other, and the round counts have been chosen so that the best attacks we are aware of fall well short of the full primitive. A detailed analysis is given in Section 7. We invite the cryptographic community to scrutinize these choices, and we expect to revisit the parameters if cryptanalytic progress warrants it.

2 Parameters and Notations

2.1 Recommended Parameters

Table 1 lists our recommended instances for QSH hash family and specifies their parameters, including the word size w , the state size h , the message block size m , the size $m/2$ of the tree-internal chaining values, and the digest size n . The list is sorted by digest length: the primary recommendation is QSH-512, which is intended as the default choice for general-purpose use. QSH-768 and QSH-1024 are provided for applications that require longer digests or higher security levels, and use a wider 64-bit word together with a larger 4096-bit state.

All three variants are built from the same compression function with the underlying permutation CHACHA BAHRU and share a common tree-based mode of operation; they differ only in the parameters listed in Table 1 and in the number of rounds of CHACHA BAHRU (see Section 3).

Table 1: Width parameters of the three QSH variants, where CV denotes chaining value in tree-based construction.

Variant	word w	state h	message block m	CV size $m/2$	digest n
QSH-512	32	2048	1024	512	512
QSH-768	64	4096	2048	1024	768
QSH-1024	64	4096	2048	1024	1024

2.2 Notations and Operations

The following parameters are used throughout the QSH specifications.

Word	A binary string of length w , i.e., an element of $\{0, 1\}^w$.
$A[i]$	The i^{th} bit in the word A . That is, a w -bit word A is represented as $A = A[0] \parallel A[1] \parallel \cdots \parallel A[w-1]$.

w	The width of a word, in bits. $w \in \{32, 64\}$.
d	The dimension of the space of 4-tuple (quadruple) words. A d -dimensional space contains 4^d words.
h	Number of bits in a state value. $h = 4^d \times w$; concretely, $h = 2048$ for $w = 32$ and $h = 4096$ for $w = 64$.
u	Number of words in a state value. $u = h/w = 4^d$; concretely $u = 64$ when $d = 3$.
n	Number of bits in the final message digest. $n \in \{512, 768, 1024\}$, depending on the QSH variant.
$H^{(i)}$	The i^{th} state value during chunk processing, of 4^d words.
$H_j^{(i)}$	The j^{th} word of $H^{(i)}$, $H^{(i)} = H_0^{(i)} \parallel H_1^{(i)} \parallel \dots \parallel H_{u-1}^{(i)}$.
M	Message to be hashed.
ℓ	Length of the message M , in bits.
m	Number of bits in a message block. $m = h/2$; concretely $m = 1024$ for $w = 32$ and $m = 2048$ for $w = 64$.
v	Number of words in a message block. $v = m/w = u/2$; concretely $v = 32$ when $d = 3$.
$M^{(i)}$	Message block i , of m bits.
$M_j^{(i)}$	The j^{th} word of $M^{(i)}$, $M^{(i)} = M_0^{(i)} \parallel M_1^{(i)} \parallel \dots \parallel M_{v-1}^{(i)}$.
N	Number of blocks in the padded message.
ρ	Number of blocks per chunk. $\rho = 16$.
Π	Number of chunks in the padded message. $\Pi = \lceil N/\rho \rceil$.
$flag$	Domain-separation flag word (Section 2.3).

The following operations are used in the QSH specifications.

$\boxplus$	Word-wise addition modulo 2^w .
$\lll$	Bitwise left-rotation operation.
$\oplus$	Bitwise XOR (“exclusive-OR”) operation.
$ $	Bitwise OR (“inclusive-OR”) operation, used to combine flag values.
$\parallel$	Concatenation operation.
$\text{trunc}_k(\cdot)$	Truncation that keeps the first k bits of a bit string.

2.3 Domain-separation Flags

Each call of the compression function $F_{d,w}$ is tagged by a w -bit *flag* word that indicates the role of the call within the QSH tree, see Section 4. The four admissible flag values, listed in Table 2, are mutually disjoint single-bit values; the *flag* of any particular call is the bitwise OR of the applicable values. A call with no applicable flag uses $flag = 0$.

Table 2: Domain-separation flag values.

Flag	Value (as w -bit word)
CHUNK_START	2^0
CHUNK_END	2^1
PARENT	2^2
ROOT	2^3

3 The CHaCha BAHRU Permutation

This section present the specification of CHaCha BAHRU, the unkeyed permutation that underlies the proposed hash algorithms QSH. CHaCha BAHRU is a family of ARX-based permutations parameterized by the word size $w \in \{32, 64\}$, and the parameters are summarized in Table 3. Unless otherwise specified, the description in this section applies to both instances.

The design follows a single guiding idea. ChaCha organises its state as a two-dimensional 4×4 array of words and alternates an ARX mixing step along the columns with the same step along the diagonals. We keep this column/diagonal rhythm but lift the state from a square to a *d -dimensional cube of words*, and we use the three-dimensional instance ($d = 3$) as the permutation of QSH. Two components suffice to describe the whole construction: a fixed transformation G that mixes four words at one time (Section 3.1), and a cheap word shuffle P_{16} that re-aligns the cube between mixing passes (Section 3.2). A round $R_{d,w}$ interleaves these two along every axis of the cube (Section 3.3); the permutation $E_{d,w}$ is a fixed number of such rounds followed by a final mixing-only step (Section 3.4).

3.1 Transformation function G on Word-Quadruple

The function G used as a component of CHaCha BAHRU transforms a quadruple of words, i.e. 4-tuple (a, b, c, d) , to another quadruple. And $G(a, b, c, d)$ is defined as the following procedure:

$$\begin{aligned}
 a &= a + (b \lll \sigma_1), & d &= (d \oplus a) \lll \tau_1, \\
 c &= c + (d \lll \sigma_2), & b &= (b \oplus c) \lll \tau_2, \\
 a &= a + (b \lll \sigma_3), & d &= (d \oplus a) \lll \tau_3, \\
 c &= c + (d \lll \sigma_4), & b &= (b \oplus c) \lll \tau_4,
 \end{aligned}$$

where rotation amounts $(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ and $(\tau_1, \tau_2, \tau_3, \tau_4)$ are dependent on the size of word w , and are given by $(\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (1, w/2 - 1, 0, w/4)$ and $(\tau_1, \tau_2, \tau_3, \tau_4) = (w/2, w/2 - 4, w/4, w/4 - 1)$. Each of the eight equations is an invertible update of one word, so G is a bijection; its inverse runs the eight ones in reverse. Its construction is shown in Figure 1 for the two instantiations.

G is the extension and generalization on w -bit word of the quarter-round function of ChaCha [9] where operations are on 32-bit word. When ChaCha adds

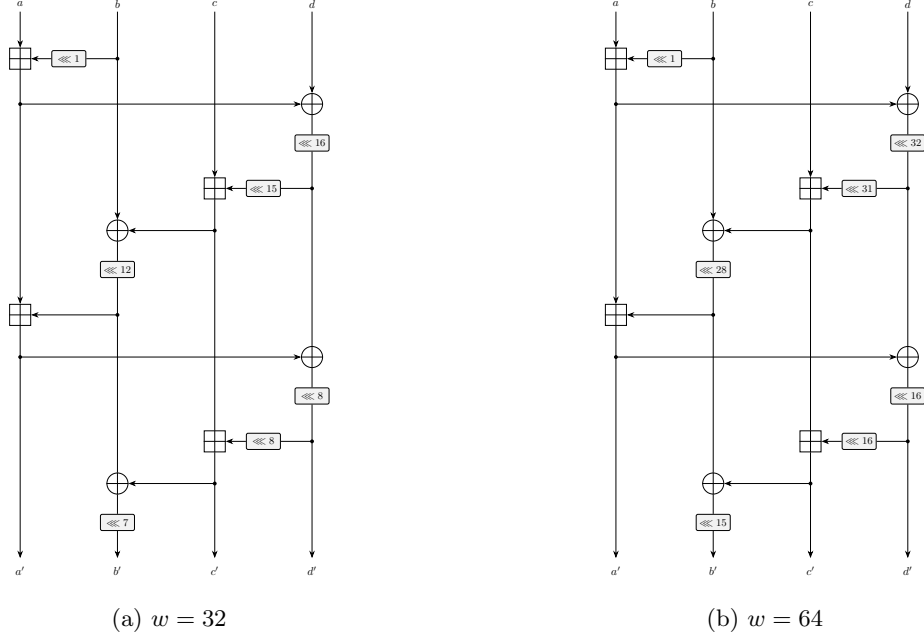


Figure 1: The word-quadruple transformation G for the two instantiations $w = 32$ and $w = 64$. The two circuits share an identical structure; only the rotation numbers differ.

two input words directly, $a \leftarrow a + b$, we add a *rotated* word, $a \leftarrow a + (b \lll \sigma_i)$. The reason is purely cryptanalytic: the direct form admits differences that are invariant under the surrounding rotations and therefore propagate through the quarter-round with probability one, and three such characteristics exist in the difference-pattern table of the original ChaCha quarter-round. Inserting a rotation on the addend breaks this invariance and removes the probability-one patterns, as verified in Section 7.3. The modification costs nothing in practice, since “add a rotated word” compiles to a single instruction on the platforms.

The rotation numbers are chosen to be simultaneously cheap in implementation and diffusion-friendly. For $w = 32$ the τ -amounts (16, 12, 8, 7) are exactly those of ChaCha20, and the σ -amounts (1, 15, 0, 8) coincide with the inner rotations of the Alzette ARX-box [7]; both have been studied extensively and implement well across architectures. For $w = 64$ we take the natural scaling $(\tau_1, \tau_2, \tau_3, \tau_4) = (32, 28, 16, 15)$ and $(\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (1, 31, 0, 16)$. The constants are not arbitrary: $\tau_4 = w/4 - 1$ is odd and coprime to w for both word sizes, which (as shown in Section 7.3.3) prevents any single rotation from making G transparent to a rotational pair.

3.2 Linear Layer : Shuffle P_{16}

When G mixes a quadruple, the shuffle P_{16} moves *between* quadruples so that the next application of G takes a *fresh* combination of words. It plays the role those of the column-to-diagonal re-indexing in ChaCha and ShiftRows step in AES [20]: a simple permutation of positions, with no arithmetic.

P_{16} acts on a two-dimensional 4×4 array $A = (a_{i,j})_{i,j \in \{0,1,2,3\}}$, where each element represents one word. The permutation operates in a row-wise manner: the first row remains unchanged, while the i -th row is cyclically shifted to the left by i word positions. Formally, the permutation P_{16} maps the input array $A = (a_{i,j})$ to the output array $A' = (a'_{i,j})$ defined as

$$a'_{i,j} = a_{i, (j+i) \bmod 4}, \quad \text{for } i, j \in \{0, 1, 2, 3\}.$$

Equivalently, the permutation can be written explicitly as

$$P_{16}(A) = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} \\ a_{2,2} & a_{2,3} & a_{2,0} & a_{2,1} \\ a_{3,3} & a_{3,0} & a_{3,1} & a_{3,2} \end{pmatrix}, \text{ where } A = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix}.$$

Cheap as it is, P_{16} is what couples otherwise independent G calls into a diffusing network. Sandwiching one column pass of G between P_{16} and P_{16}^{-1} turns it into a pass of G along the *diagonals* of the array, like ChaCha turns its column round into a diagonal round by re-indexing. This is the mechanism the round function uses below.

3.3 Round Function $R_{d,w}$

The round function, denoted by $R_{d,w}$, takes $4^d w$ -bit states as input and output. First, the quadruple transformation in Section 3.1 is iteratively applied to 4 words in parallel of $(d-1)$ -dimensional space until d dimensions exhausted; Then, use a two-dimensional permutation P_{16} of Section 3.2 between current and next dimensions.

Write the $4^d w$ -bit input to $R_{d,w}$ as $A = A_0 \parallel A_1 \parallel \dots \parallel A_{4^d-1}$ with each A_i a w -bit word, and view it as a d -dimensional array indexed by $\mathbb{Z}_4^d$,

$$A = (A_{(x_0, x_1, \dots, x_{d-1})})_{(x_0, \dots, x_{d-1}) \in \mathbb{Z}_4^d},$$

where each entry $A_{(x_0, \dots, x_{d-1})}$ is a w -bit word. And the linear index and the coordinate index are related by

$$i = x_0 \cdot 4^{d-1} + \sum_{j=1}^{d-1} x_j \cdot 4^{j-1}, \quad A_i = A_{(x_0, x_1, \dots, x_{d-1})}.$$

For an axis $k \in \{0, 1, \dots, d-1\}$ and any fixed choice of the remaining coordinates, the *line along axis k* is the 4-tuple obtained by letting x_k range over $\mathbb{Z}_4 = \{0, 1, 2, 3\}$,

$$A_{(x_0, \dots, x_{k-1}, *, x_{k+1}, \dots, x_{d-1})} := (A_{(x_0, \dots, x_{k-1}, t, x_{k+1}, \dots, x_{d-1})})_{t \in \mathbb{Z}_4}.$$

Such a line is precisely a quadruple of words, so it can be fed to G .

One round. A round first applies G to every line along axis 0. It then walks through the remaining axes $i = 1, \dots, d-1$: for each axis it applies P_{16} to the (x_{i-1}, x_i) -planes, applies G along axis $i-1$, and undoes the shuffle with P_{16}^{-1} . As noted here, this P_{16} -sandwich is exactly “apply G along the diagonals of the (x_{i-1}, x_i) -plane”, so axis i is coupled to axis $i-1$ without any new arithmetic.

The corresponding output is denoted by $A' = A'_0 \parallel A'_1 \parallel \dots \parallel A'_{4^d-1}$. And The computation of $A' = R_{d,w}(A)$ is given in Algorithm 1.

Algorithm 1 Round function $R_{d,w}$

```

1: for  $(x_1, x_2, \dots, x_{d-1}) \in \mathbb{Z}_4^{d-1}$  do
2:    $(A_{(x_0, x_1, \dots, x_{d-1})})_{x_0 \in \{0,1,2,3\}} = G \left( (A_{(x_0, x_1, \dots, x_{d-1})})_{x_0 \in \{0,1,2,3\}} \right)$  ▷
   column transformation layer
3: end for
4: for  $1 \leq i \leq d-1$  do
5:   for  $(x_0, \dots, x_{i-2}, x_{i+1}, \dots, x_{d-1}) \in \{0,1,2,3\}^{d-2}$  do
6:      $(A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, x_{i+1}, \dots, x_{d-1})})_{(x_{i-1}, x_i) \in \{0,1,2,3\}^2}$  =
      $P_{16} \left( (A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, x_{i+1}, \dots, x_{d-1})})_{(x_{i-1}, x_i) \in \{0,1,2,3\}^2} \right)$ 
7:     end for
8:     for  $(x_0, \dots, x_{i-2}, x_i, \dots, x_{d-1}) \in \{0,1,2,3\}^{d-1}$  do
9:        $(A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, \dots, x_{d-1})})_{x_{i-1} \in \{0,1,2,3\}}$  =
        $G \left( (A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, \dots, x_{d-1})})_{x_{i-1} \in \{0,1,2,3\}} \right)$ 
10:      end for
11:      for  $(x_0, \dots, x_{i-2}, x_{i+1}, \dots, x_{d-1}) \in \{0,1,2,3\}^{d-2}$  do
12:         $(A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, x_{i+1}, \dots, x_{d-1})})_{(x_{i-1}, x_i) \in \{0,1,2,3\}^2}$  =
         $P_{16}^{-1} \left( (A_{(x_0, \dots, x_{i-2}, x_{i-1}, x_i, x_{i+1}, \dots, x_{d-1})})_{(x_{i-1}, x_i) \in \{0,1,2,3\}^2} \right)$ 
13:      end for
14:    end for
15: return  $A'$ 

```

The three-dimensional case. When $d = 3$, let $(x_0, x_1, x_2) \in \{0,1,2,3\}^3$ indexing total $4 \times 4 \times 4$ words of states, as illustrated in Figure 2, a round consists of three passes of 16 parallel G calls each:

1. *Columns.* G is applied to every line along x_0 with all constant (x_1, x_2) traversed, see subfigure (a) of Figure 2.
2. $i = 1$. G couples axes (x_0, x_1) : within each plane spanned by (x_0, x_1) , G runs along the diagonals via the P_{16} -sandwich, see subfigure (b) of Figure 2.

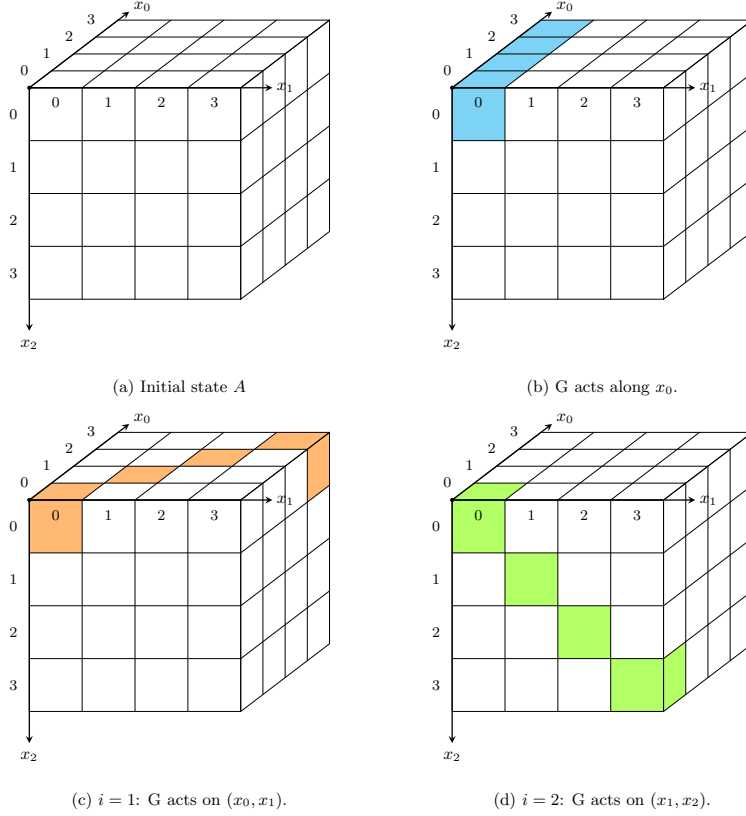


Figure 2: Illustration of $R_{3,w}$.

3. $i = 2$. G couples axes (x_1, x_2) in the same diagonal fashion, see subfigure (c) of Figure 2.

To facilitate the comprehension, the round of $R_{3,w}$ is shown in Figure 3, including the three passes. Because the sixteen G calls of each pass are independent, a round exposes 16-fold parallelism three times over—the property that the vectorized implementations of Section 8 exploit.

Also, at Table 15, 16 and 17 in Appendix A, we summarize the map between three-dimensional index (x_0, x_1, x_2) and individual indices A_i for each loop (G layer) of Algorithm 1 for further implementation and verifications.

3.4 Permutation $E_{d,w}$ and CHACHA BAHRU

$E_{d,w}$ is a permutation based on ARX operations. It is constructed from $n_r = d \cdot (3w/32)$ rounds of $R_{d,w}$, plus an additional column transformation layer. The $4^d w$ -bit input and output are denoted as S and S' , respectively. Let each A^r

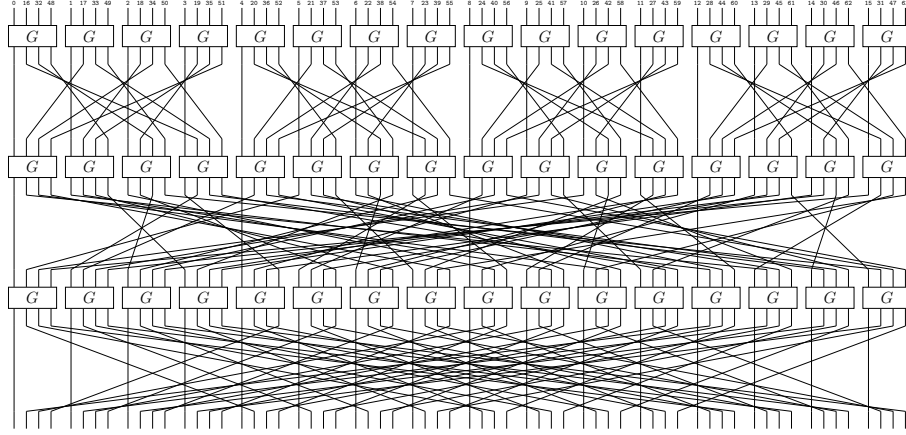


Figure 3: Round function $R_{3,w}$ in a line.

denote a $4^d w$ -bit string, $0 \leq r \leq n_r + 1$, and $A^r = (A_0^r \| A_1^r \| \dots \| A_{4^d-1}^r)$, where each A_i^r denotes a w -bit word. Let $R_{d,w}^*$ denote the round function $R_{d,w}$ with only the column transformation layer. The computation of $S' = E_{d,w}(S)$ is specified in Algorithm 2.

Algorithm 2 Permutation $E_{d,w}$

- 1: group the bits of S into 4^d consecutive w -bit words to obtain A^0 .
 - 2: **for** $0 \leq r \leq n_r - 1$ **do**
 - 3: $A^{r+1} = R_{d,w}(A^r)$;
 - 4: **end for**
 - 5: $A^{n_r+1} = R_{d,w}^*(A^{n_r})$ ▷ final column-only round
 - 6: **return** $S' = A^{n_r+1}$
-

We focus on the case of $E_{3,w}$ with $d = 3$, dubbed CHACHA BAHRU- w , which is used as the underlying permutation for the hash. We use the instance of CHACHA BAHRU- w when $w = 32$ and 64, whose parameters are listed in Table 3.

Table 3: Parameters of CHACHA BAHRU when $w = 32/64$.

Instance	Rounds	Number of words	Word size w	State size
CHACHA BAHRU-32	10	64	32	2048
CHACHA BAHRU-64	19	64	64	4096

4 Compression Function via Modular Addition

The compression function is adapted from JH’s compression function [50], where the message absorption is injected by modular addition instead of XOR. The compression function $F_{d,w}$ takes an $(h/2)$ -bit message block $M^{(i)}$ and an h -bit state value $H^{(i-1)}$, and produces a new h -bit state value

$$H^{(i)} = F_{d,w}(M^{(i)}, H^{(i-1)}),$$

where $h = w \cdot 2^{2d}$. $F_{d,w}$ is constructed from the permutation $E_{d,w}$ of Section 3.4; and the construction is shown in Figure 4. According to the definition of $E_{d,w}$ (the column transform layer of $R_{d,w}$ is always orthogonal to input and output), the input to every first-layer quarter-round G function is affected by two message words, and the output of every last-layer quarter-round G function is modular added with two message words.

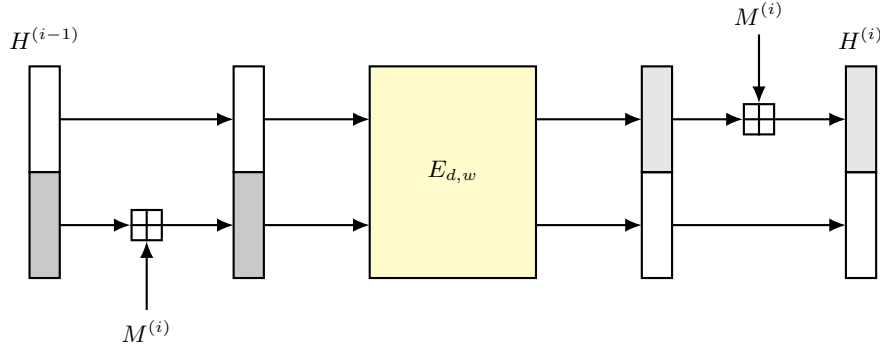


Figure 4: Compression function $F_{d,w}$.

Therefore, the message processing phase iteratively applied the compression function to inject all message blocks, as illustrated in Figure 5.

Remark 1. On domain separation for $F_{d,w}$. Each call of $F_{d,w}$ is tagged by a w -bit *flag* word (Section 2.3) that indicates the role of the call within the QSH tree. Inside $F_{d,w}$, the *flag* is mixed into the last word of the state by $S_0^{(i-1)} = S_0^{(i-1)} \oplus \text{flag}$ right before $E_{d,w}$ is applied.

$F_{3,32}$ and $F_{3,64}$. $F_{3,w}$ ($w \in \{32, 64\}$) is the compression function used by QSH, built from the keyless permutation $E_{3,w}$ of Section 3.4, say CHACHA BAHRU- w . With $d = 3$ we have $h = 4^3 w$, so CHACHA BAHRU- w acts on an h -bit state: $h = 2048$ for $w = 32$ and $h = 4096$ for $w = 64$. $F_{3,w}$ compresses an $(h/2)$ -bit message block $M^{(i)}$ and an h -bit state value $H^{(i-1)}$ into the h -bit state value $H^{(i)} = F_{3,w}(M^{(i)}, H^{(i-1)})$, as specified in Algorithm 3. The *flag* word used in Step 5 of the algorithm is set by the calling algorithm based on the position of the call in the QSH tree (see Section 5.3, Section 5.4).

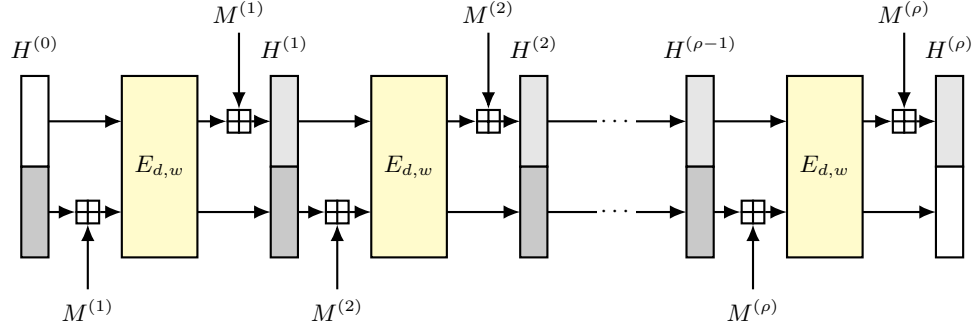


Figure 5: The message processing phase with injecting ρ blocks.

Algorithm 3 Compression function $F_{3,w}$

Require: state value $H^{(i-1)} \in \{0, 1\}^h$; message block $M^{(i)} \in \{0, 1\}^{h/2}$; and ambient $flag \in \{0, 1\}^w$ from the caller.

Ensure: state value $H^{(i)} \in \{0, 1\}^h$

- 1: **for** $0 \leq j \leq v-1$ **do**
 - 2: $S_j^{(i-1)} = H_j^{(i-1)}$.
 - 3: $S_{v+j}^{(i-1)} = H_{v+j}^{(i-1)} \boxplus M_j^{(i)}$. ▷ inject $M^{(i)}$ into lower half
 - 4: **end for**
 - 5: $S_0^{(i-1)} = S_0^{(i-1)} \oplus flag$. ▷ set domain-separation flag
 - 6: $S^{(i)} = E_{3,w}(S^{(i-1)})$.
 - 7: **for** $0 \leq j \leq v-1$ **do**
 - 8: $H_j^{(i)} = S_j^{(i)} \boxplus M_j^{(i)}$. ▷ inject $M^{(i)}$ into upper half
 - 9: $H_{v+j}^{(i)} = S_{v+j}^{(i)}$.
 - 10: **end for**
 - 11: **return** $H^{(i)}$
-

5 Specifications of QSH Algorithms

The family of QSH algorithms consists of four steps: padding the message M (Section 5.1), parsing the padded message into message chunks (Section 5.2), computing the hash value for each message chunk in parallel (Section 5.3), and generating the message digest from the hash values of all chunks by tree structure (Section 5.4).

5.1 Padding Message

The message M is padded to a multiple of m bits, where $m = 1024$ for QSH-512 and $m = 2048$ for QSH-768 and QSH-1024. Suppose the length of M is ℓ bits. Append a single 1 bit to the end of M , followed by $(2m - 129 - (\ell \bmod m))$ zero

bits, then append the 128-bit binary representation of ℓ in little-endian order. Thus at least m additional bits are appended to the message M .

5.2 Parsing Padded Message into Chunks

After a message has been padded, it is represented as N m -bit blocks, $M^{(1)}, M^{(2)}, \dots, M^{(N)}$. The first ρ blocks are parsed into the first chunk $C^{(1)} = M^{(1)} \| M^{(2)} \| \dots \| M^{(\rho)}$, then the second chunk $C^{(2)} = M^{(\rho+1)} \| M^{(\rho+2)} \| \dots \| M^{(2\rho)}$, and so on, until the last chunk $C^{(\Pi)} = M^{((\Pi-1)\times\rho+1)} \| \dots \| M^{(N)}$ such that the last chunk may be shorter, but not empty, unless the entire input is empty.

5.3 Computing Chunk Hash Value

The chunk hash value of chunk $C^{(t)}$ ($1 \leq t \leq \Pi$) is generated by iteratively applying $F_{3,w}$ to its L message blocks, where $L = \rho$ for $1 \leq t < \Pi$ and $1 \leq L \leq \rho$ for the last chunk $t = \Pi$. We index blocks within a chunk by $1 \leq i \leq L$; $H^{(0)}$ denotes the variant-specific initial state value defined below, and $H^{(i)}$ ($1 \leq i \leq L$) is the running state value inside the chunk. The chunk chaining value is the truncation of $H^{(L)}$ to $(m/2)$ bits, computed as specified in Algorithm 4.

Variant-specific initial state value $H^{(0)}$. Let $H^{(-1)}$ be the raw Initial Vector (IV): $H_0^{(-1)}$ takes the variant constant 17, 1, or 7 for QSH-512, QSH-768, and QSH-1024, respectively, and $H_j^{(-1)} = 0$ for all $1 \leq j \leq u-1$. The initial state value is the precomputed constant

$$H^{(0)} = F_{3,w}(0^m, H^{(-1)}),$$

where 0^m denotes the all-zero message block. $H^{(0)}$ is computed once and reused as the starting state value of every chunk and every parent.

Algorithm 4 Chunk processing

Require: message blocks $M^{(1)}, \dots, M^{(L)}$ of chunk $C^{(t)}$; a Boolean *is_only_chunk* indicating whether the message contains exactly one chunk

Ensure: chunk chaining value $cv^{(t)} \in \{0, 1\}^{m/2}$

```

1: for  $1 \leq i \leq L$  do
2:    $flag = 0$ 
3:   if  $i = 1$  then  $flag = flag \mid \text{CHUNK\_START}$ 
4:   end if
5:   if  $i = L$  then  $flag = flag \mid \text{CHUNK\_END}$ 
6:   end if
7:   if  $i = L$  and is_only_chunk then  $flag = flag \mid \text{ROOT}$ 
8:   end if
9:    $H^{(i)} = F_{3,w}(M^{(i)}, H^{(i-1)})$   $\triangleright$  absorb  $M^{(i)}$  with the current flag
10: end for
11: return  $\text{trunc}_{m/2}(H^{(L)})$ 

```

5.4 Generating Message Digest

The message digest is generated by connecting all the chunk chaining values via a binary tree structure. After chunk processing (Section 5.3), each chunk $C^{(t)}$ has a chaining value $cv^{(t)} \in \{0,1\}^{m/2}$ (chunk hash value). These leaf chaining values are assembled bottom-up with parent nodes (assuming at least two chunks), where each parent has exactly two children. For each parent node, $F_{3,w}$ is applied to derive a new $m/2$ -bit chaining value from its two child chaining values, as specified in Algorithm 5.

Algorithm 5 Parent chaining value generation

Require: chaining values of left and right children $cv_L, cv_R \in \{0,1\}^{m/2}$; a Boolean is_root indicating whether this parent is the root of the tree.

Ensure: parent chaining value $cv \in \{0,1\}^{m/2}$.

```

1:  $flag = \text{PARENT}$ 
2: if  $is\_root$  then  $flag = flag \mid \text{ROOT}$ 
3: end if
4:  $M = cv_L \parallel cv_R$  ▷ form an  $m$ -bit message from the two child CVs
5:  $I^{(0)} = H^{(0)}$  ▷ precomputed initial chaining value (Section 5.3)
6:  $I^{(1)} = F_{3,w}(M, I^{(0)})$  ▷ compress with the current  $flag$ 
7: return  $\text{trunc}_{m/2}(I^{(1)})$ 

```

For uniqueness of message digest, the tree structure should be determined for the uniqueness. If there is only one chunk, that chunk is the root node and only node of the tree. Otherwise, the chunk is assembled with parent nodes, and each parent node has exactly two children by the following way. The tree structure are constructed by two rules as follows:

- Left subtrees are full. Each left subtree is a complete binary tree, with all its chunks at the same depth, and a number of chunks equal a power of 2.
- Left subtrees are big. Each left subtree contains a number of chunks greater than or equal to the number of chunks in its sibling right subtree.

For example, trees from one to four chunks have the structure shown in Figure 6. Each chunk_{*i*} denotes a contiguous message blocks, and the corresponding cv_i is the chaining value produced by compressing chunk_{*i*}. Internal nodes (unlabelled) are parent chaining values; the topmost node is the root.

The compression function is used to derive a chaining value from each chunk (Algorithm 4) and for parent nodes (Algorithm 5). Let $cv_{\text{root}} \in \{0,1\}^{m/2}$ denote the chaining value of the root node:

- if there is exactly one chunk ($\Pi = 1$, hence $is_only_chunk = \text{true}$), cv_{root} is the output of Algorithm 4 for that single chunk;
- otherwise, cv_{root} is the output of the topmost call of Algorithm 5, with $is_root = \text{true}$.

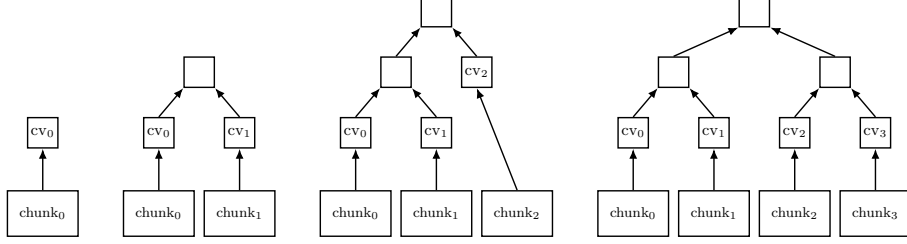


Figure 6: Example tree structures from 1 to 4 chunks.

Finally, the QSH- n message digest is obtained by truncating the root chaining value to the variant’s output length:

$$\text{Digest} = \text{trunc}_n(\text{cv}_{\text{root}}),$$

where $n \in \{512, 768, 1024\}$ for QSH-512, QSH-768, QSH-1024, respectively; the digest is encoded as a sequence of words in little-endian order. Note that $n = (m/2)$ for QSH-512 and QSH-1024, QSH-768 actually shortens the root chaining value to 768 bits.

6 Design Rationale and Features

Choice of ARX Design. Following a now-common design philosophy, the underlying permutation is constructed entirely from Addition, Rotation, and XOR (ARX) operations. ARX designs admit a simple description, and efficient implementation in software on a wide range of platforms. The ARX design paradigm has been extensively applied in many well-known ciphers, such as stream cipher: ChaCha [9], Salsa20 [10]; block ciphers: SPECK [6], LEA [30]; hash functions: SHA-1 [43], SHA-2 [43], BLAKE family [4] [5] [44], and Skein [24].

ChaCha together with Poly-1305 are widely employed in practice. Also, the hash BLAKE family use ChaCha’s permutation as core function. A distinctive feature of ChaCha [9] is that its state is organized as a two-dimensional 4×4 array of 32-bit words, and the round function alternates between operations along the columns and along the diagonals of this array. We adopt the same principle in QSH, but generalize the state to a d -dimensional cube ($d \geq 2$), and use the three-dimensional instance, called as CHACHA BAHRU. Concretely, for QSH-512 the 2048-bit state is partitioned into $4^3 = 64$ words of $w = 32$ bits, arranged as a $4 \times 4 \times 4$ cube; the round function operates along the three axes of the cube in turn. The three-dimensional structure preserves the simplicity and software efficiency of the ChaCha-style round, while providing a wider state and richer diffusion patterns.

6.1 Principles of CHACHA BAHRU

The prime requirement is that the state size of permutation is large enough to ensure that the requirement of 512/1024-bit hash length could be satisfied. Then, besides security and efficient implementation, the main goal when designing the permutation was simplicity. First, we give the intuitive principles for the underlying components of permutation.

Quarter-word transformation G . G is the ChaCha-style ARX function transposed to arbitrary word width w (Section 3.1); the one departure from that of ChaCha [9] is that each modular addition adds a *rotated* operand rather than a bare one. This simple but useful change is dictated by three design goals, all of which the resulting G provably meets.

(i) *No probability-one differential.* For every instantiated width G has no non-trivial weight-0 differential trail. The original ChaCha quarter-round has three non-trivial weight-0 differential trails, such trail chain across rounds and would erode any differential bound.

(ii) *Width-independent propagation structure.* The difference support of G is one and the same for all instantiated widths and its differential branch number is large enough.

(iii) *Provable bound by counting active G .* Goals (i)–(ii) make the differential analysis of $E_{3,w}$ structurally elementary: since each active G costs at least one unit of weight and the propagation rules do not depend on w , a lower bound on the number of active G across the rounds of $E_{3,w}$ immediately gives a width-uniform lower bound on the weight of any differential trail.

Proposition 1. Let G be the quarter-word transformation of Section 3.1 with rotation amounts $(\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (1, \frac{w}{2} - 1, 0, \frac{w}{4})$ and $(\tau_1, \tau_2, \tau_3, \tau_4) = (\frac{w}{2}, \frac{w}{2} - 4, \frac{w}{4}, \frac{w}{4} - 1)$. Then:

- (1) Every active G contain at least one non-trivial modular addition, that is, contributes at least one bit of differential weight. So, *differential pattern distribution*(DPDT, see Definition 1) of G has no non-trivial weight-0 entry.
- (2) The support S_G is identical for $w \in \{16, 32, 64\}$, and the cell weights are non-decreasing in the width: for every $(e, e') \in S_G$ and $w_1 \geq w_2$, $\text{wt}_{w_1}(e, e') \geq \text{wt}_{w_2}(e, e')$. In particular ChaCha Bahru-32 and ChaCha Bahru-64 share one and the same word-level trail model, and the per- G weights of ChaCha Bahru-64 dominate those of ChaCha Bahru-32.
- (3) The differential branch number of G is $\min_{e \neq 0, e' \in S_G(e)} (|e| + |e'|) = 3$.

S_G is the set of pairs (e, e') of input and output activity patterns for which some differential of G with that activity pattern has non-zero probability.

The proof is given in Appendix B.

The rotation numbers $(\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (1, \frac{w}{2} - 1, 0, \frac{w}{4})$ coincide with those of Alzette [7] at $w = 32$ for efficient cross-platform implementation, while $(\tau_1, \tau_2, \tau_3, \tau_4) = (\frac{w}{2}, \frac{w}{2} - 4, \frac{w}{4}, \frac{w}{4} - 1)$ are inherited from ChaCha at $w = 32$ and extended naturally to $w = 64$. Add-of-rotation executes in a single cycle on ARM, so the change is essentially free in software.

Linear layer P_{16} . P_{16} is a plain shuffle of the 16 words of a 4×4 slice (equivalent to the ShiftRows of AES [20]), hence costless in both software and hardware. Its design purpose is cross-column diffusion: it maps the four words of every column to four *distinct* columns. Consequently the active output words of any single G are never re-collided into a common column but are dispersed to that many distinct columns of the next G -layer; together with the intra-column mixing of G (branch number 3) this disperses and amplifies activity along the three axes of the cube, and is what drives the growth of the active G count underlying Table 19. We deliberately keep P_{16} a mere permutation rather than an MDS-type layer: a stronger per-step branch number is not needed, since the required margin already follows from the multi-round active- G growth, and a shuffle preserves the implementation simplicity that is a central goal of QSH.

Permutation $E_{3,w}$. $E_{3,w}$ is built from $n_r = d \cdot (3w/32)$ full rounds of $R_{d,w}$ plus one final column-only step. For $d = 3$ this yields $n_r = 9$ full rounds at $w = 32$ and $n_r = 18$ full rounds at $w = 64$. Each round visits the three axes of the 3-dimensional state in turn; the round count $(n_r + 1)$ is therefore a multiple of 3 modulo the trailing column step, ensuring symmetric exposure of all axes.

6.2 Features

We summarize the main features of QSH below.

- **CHACHA BAHRU with a Provable Security Bound.** We propose a new permutation, CHACHA BAHRU- w , built from an enhanced quarter-round function G in which every modular addition absorbs a *rotated* operand. This single change removes the probability-one differentials of the original ChaCha quarter-round and lets us prove explicit lower bounds on the number of active G 's, and hence on the differential trail weights. Also, by lifting ChaCha's two-dimensional column/diagonal mixing to a three-dimensional cube of words, CHACHA BAHRU operates on a much wider and flexibility state, while inheriting the well-studied diffusion and non-linearity of the ChaCha family.
- **Message Absorption via Modular Addition.** Instead of the usual XOR, QSH absorbs each message block by *modular addition*, and injects it *twice*—into the lower half of the state before the permutation and into the upper half afterwards, in a JH-style keying. This double, addition-based injection rapidly raises the algebraic degree of the compression function without changing the underlying permutation, providing a robust line of defence against algebraic and structural attacks on the mode.

Table 4: Single-message throughput of QSH as a function of core count, on an Intel Xeon (2.80 GHz, 12 vCPUs), and messages are 16 MiB.

Threads	Throughput (MB/s)			Speedup (QSH-512)
	QSH-512	QSH-768	QSH-1024	
1	327.2	214.8	215.4	1.00×
2	604.8	407.8	406.9	1.85×
4	1103.0	741.0	737.4	3.37×
8	1384.9	874.2	877.7	4.23×
12	1855.2	1167.7	1150.3	5.67×

- **Two-Level Parallelism of the Tree Hash.** QSH exposes parallelism at two orthogonal, composable levels. *(i) Data level (within a permutation):* every layer of CHACHA BAHRU’s round function applies sixteen *independent* copies of G , which map directly onto SIMD lanes; this vectorization accelerates *every* invocation, including short messages. *(ii) Task level (across chunks):* the tree mode decomposes a message of L bytes into $\lceil L/\text{chunk} \rceil$ *independent* chunks that can be hashed concurrently. In contrast to inherently serial Merkle–Damgård or sponge constructions—where a single long message admits no internal parallelism—QSH’s throughput on long inputs scales with the available SIMD width and core count.

A throughput that scales with hardware parallelism. The two levels of parallelism combine *multiplicatively*. Writing r_1 for the single-lane (scalar) rate, W for the SIMD width (lanes advanced per instruction), and P for the number of cores, the sustained throughput of QSH on a long message behaves as

$$\text{Throughput} \approx r_1 \cdot W \cdot P,$$

up to the point where memory bandwidth saturates. The factor W stems from the sixteen independent copies of G in every layer of CHACHA BAHRU (filled by SIMD lanes), and the factor P from the $\lceil L/\text{chunk} \rceil$ independent leaf chunks (distributed over cores)—and, crucially, *both factors apply to a single message*. This is a structural, generational advantage over today’s dominant serial designs: Merkle–Damgård hashes such as SHA-2 and Sponge hashes such as SHA-3 chain their compression calls, so hashing one long message is inherently sequential and cannot be sped up by wider vectors or more cores. QSH instead turns a long hash into many independent compressions, so its throughput grows with the parallel resources of the platform—from a single SIMD core to wide-vector, many-core processors—which is exactly the regime of bulk file hashing, data authentication, and Merkle-tree workloads that motivate QSH. Table 4 confirms this scaling empirically: distributing the leaf chunks over cores multiplies the single-thread SIMD rate almost linearly.

7 Security Claim and Analysis

The security claims of QSH is presented in Section 7.1. The analysis of structure used in QSH is given in Section 7.2. In Section 7.3, we analyze the cryptographic properties of the permutation CHACHA BAHRU, including differential and linear trail bounds, and rotation properties. Also, in Section 7.4, we give a preliminary analysis of resistance of QSH against the collision, (second-)preimage attacks. In addition, the analysis of several specific attacks on QSH is given in Section 7.5, together with the statistical-randomness test.

7.1 Security Claim

For every variant QSH- n ($n \in \{512, 768, 1024\}$) we claim the security strengths shown in Table 5 for the classic and quantum settings, respectively. The classical security of QSH is derived from tree hashing mode under the assumption that $E_{3,w}$ is a pseudo-random permutation, see Section 7.2. For the quantum setting, our claims follow the generic bounds for an ideal hash function under the standard quantum query models. (Second-)preimage resistance is governed by Grover’s search [26], whose quadratic speedup reduces the cost of finding an n -bit (second-)preimage from 2^n to $\tilde{O}(2^{n/2})$. Collision resistance is governed by the BHT algorithm [17], which finds a collision in $\tilde{O}(2^{n/3})$ quantum queries; this is tight, matching the $\Omega(2^{n/3})$ lower bound of Aaronson and Shi [1]. We therefore claim $n/3$ bits of collision security and $n/2$ bits of (second-)preimage security in the quantum setting, as listed in Table 5.

Table 5: The claimed security strengths (bits) of QSH variants, where $|\text{Digest}|$ denotes the digest length.

Variant	$ \text{Digest} $	Classical		Quantum	
		Collision	(2nd-)Preimage	Collision	(2nd-)Preimage
QSH-512	512	256	512	512/3	256
QSH-768	768	384	768	256	384
QSH-1024	1024	512	1024	1024/3	512

7.2 Analysis of the Construction

7.2.1 Tree-based construction

At the top level, QSH uses tree hashing [41, 21] in the construction. The workflow of QSH resembles a combination of JH [49] and sponge [12] (truncated output) constructions except for the message absorbing with modular addition. It is proven [21, 27] that the construction is resistant to standard cryptographic attacks, including collision and (second) preimage attacks. Also, the distinguishability of the tree hashing is studied in [19, 28, 27], i.e. it remains distin-

guishable from a random oracle when instantiated with an ideal compression function, block cipher, or permutation under the specified requirements.

As comparison, the hash algorithm BLAKE3 [44] also adopts a tree hashing structure but with Merkle–Damgård construction and Davies-Meyer structure and 512-bit ChaCha as underlying permutation, and its hash length is only 256 bits. QSH has a tree-sponge style structure and a double-injection message like JH, but it uses a new message absorbing by modular addition, which is adapted for providing a better algebraic properties. Besides, QSH has a more large and flexible underlying permutation, CHACHA BAHRU- w , e.g., CHACHA BAHRU-32/64 have 2048/4096-bit state, respectively.

7.2.2 Compression function with modular addition

The QSH compression function (see Figure 4) is constructed upon JH compression function [50]. The difference is that the message absorption mechanism is replaced by modular addition instead of XOR. The new message absorption admits good algebraic properties without increasing the cost of software implementation intuitively, higher algebraic degrees with modular addition; which follows a similar analysis of that for block cipher [8].

Also, the construction admits the standard JH security bounds [50, 14, 42], under the random-permutation assumption on $E_{3,w}$. Collisions on the full h -bit output of $F_{d,w}$ require $\Theta(2^{h/2})$ queries, and preimages require $\Theta(2^h)$ queries. Since $h = (16/3)n$ (QSH-768) or $h = 4n$ (QSH-512, QSH-1024), these bounds are at least as strong as the claimed $2^{n/2}$ collision and 2^n preimage levels, with substantial margin.

7.3 Analysis of the Permutation—ChaCha Bahru

7.3.1 Differential-trail bound property

We track the minimum weight of active G calls in any differential (pattern) trail through $E_{3,w}$ at the word level: every state word is either “active” (carries a non-zero XOR-difference) or inactive, and a G call is counted as active whenever any of its four input words is active. This rule is an over-approximation of bit-level activity, so the counts below are LOWER BOUNDS at the bit level.

Per- G differential bound. The differential probability (DP) of a function F on input differential α and output differential β is

$$\text{DP}_F(\alpha \rightarrow \beta) = \Pr_x[F(x \oplus \alpha) \oplus F(x) = \beta] \in [0, 1].$$

The Lipmaa–Moriai bound for a single modular addition $c = a + b \bmod 2^w$ gives $\text{DP}_{\text{add}}^{\max} \leq 2^{-1}$ for every non-trivial input–output differential triple [37].

Similar to difference distribution table (DDT) of an S-box [15], we introduce *difference pattern distribution table* (DPDT) of quadruple function $G : \mathbb{F}_{2^w}^4 \rightarrow \mathbb{F}_{2^w}^4$ to characterize its propagation of difference patterns over the unit — word.

Definition 1 (Difference Pattern Distribution Table). Let $e = (e_0, e_1, e_2, e_3)$, $e' = (e'_0, e'_1, e'_2, e'_3) \in \mathbb{F}_2^4$ be input difference pattern and output difference pattern of quadruple transformation $G : \mathbb{F}_{2^w}^4 \rightarrow \mathbb{F}_{2^w}^4$, respectively. The entry $dw(e, e')$ of function G 's DPDT is defined as $dw(e, e') = \log_2(dp(e, e'))$, i.e. weight of differential characteristic's probability where $dp(e, e')$ is:

$$\max_{\substack{\text{nz}(\Delta_i)=e_i, \text{nz}(\Delta'_i)=e'_i, \\ \text{for } i=0,1,2,3}} \left(\frac{|\{x = (x_0, x_1, x_2, x_3) \in \mathbb{F}_{2^w}^4 \text{ s.t. } G(x) \oplus G(x \oplus \Delta) = \Delta'\}|}{2^{4w}} \right)$$

where $\text{nz} : \mathbb{F}_{2^w} \rightarrow \mathbb{F}_2$ is non-zero indicator function, that is, $\text{nz}(x) = 1$, if $x \neq 0$, $\text{nz}(x) = 0$ if $x = 0$, and $\Delta = (\Delta_0, \Delta_1, \Delta_2, \Delta_3)$ and $\Delta' = (\Delta'_0, \Delta'_1, \Delta'_2, \Delta'_3)$.

Table 6: Difference pattern distribution table of G when $w = 32$, where '-' means no differential trail is possible at current pattern.

(e, e')	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0001	-	-	-	-	-	-	-	-	-	-	53	16	-	29	25	3
0010	-	-	-	-	-	-	-	-	-	-	-	15	-	21	-	2
0011	-	-	-	-	-	-	-	1	-	-	29	5	-	3	2	3
0100	-	-	-	70	-	83	-	27	-	37	60	16	-	16	6	3
0101	-	-	-	29	23	61	-	5	-	33	34	12	-	8	7	3
0110	-	-	-	35	-	33	-	4	-	37	37	8	21	9	6	3
0111	-	-	29	5	-	5	4	4	-	3	7	5	3	5	3	3
1000	-	-	-	72	-	81	-	38	-	-	62	22	-	32	29	5
1001	-	-	-	29	-	63	-	3	-	-	37	7	-	5	4	1
1010	-	-	-	34	-	36	-	3	4	-	28	7	-	7	6	2
1011	-	-	28	4	-	3	2	3	1	-	28	5	-	4	3	2
1100	-	-	-	68	-	77	-	3	-	37	50	4	-	5	5	2
1101	-	62	-	5	1	61	-	2	-	4	4	3	-	4	3	3
1110	-	-	-	2	-	3	1	3	-	5	8	4	4	3	5	3
1111	-	2	2	3	-	3	3	2	-	3	3	3	2	4	2	2

Based on the MILP method [25], we built the corresponding MILP models of searching differential characteristics for G , and computed the DPDT of G , given in Table 6. Among all possible entries of Table 6, the minimum one is 1 except the trivial one $0000 \rightarrow 0000$ with 0. That is, every active G contain at least one non-trivial modular addition, we obtain the *upper bound*

$$\text{DP}_G^{\max} \leq 2^{-1}.$$

After the DPDT of G , we therefore built an automatic tool to search differential pattern trails of $E_{3,w}$ by MILP, which presents differential trail bounds for $E_{3,w}$. As a result, limited by solving times of MILP models, we got the minimum weights of differential pattern trails, up to 3 rounds of $E_{3,w}$. The minimum weight of differential pattern trails are 7, 42 and 91 for 1 round, 2

Table 7: Round-by-round lower bound $LB(r)$ of probability weight in any differential (pattern) trail through ChaCha Bahru $E_{3,w}$. Rounds 1–3: proven optimum; Rounds 4–24: bound via partition.

ChaCha Bahru-32 ($n_r+1 = 10$)			ChaCha Bahru-64 ($n_r+1 = 19$)		
Round r	$LB(r)$	Partition	Round r	$LB(r)$	Partition
1	7	1	1	7	1
2	42	2	2	42	2
3	91	3	3	91	3
4	98	3 + 1	4	98	3 + 1
5	133	3 + 2	5	133	3 + 2
6	182	3 + 3	6	182	3 + 3
7	189	3 + 3 + 1	7	189	3 + 3 + 1
8	224	3 + 3 + 2	8	224	3 + 3 + 2
9	273	3 + 3 + 3	9	273	3 + 3 + 3
10*	274	3 + 3 + 3 + 1	10	280	3 + 3 + 3 + 1
			13	371	3 + 3 + 3 + 3 + 1
			14	406	3 + 3 + 3 + 3 + 2
			15	455	3 + 3 + 3 + 3 + 3
			16	462	3 + 3 + 3 + 3 + 3 + 1
			17	497	3 + 3 + 3 + 3 + 3 + 2
			18	546	3 + 3 + 3 + 3 + 3 + 3
			19*	547	3 + 3 + 3 + 3 + 3 + 3 + 1

* Final column-only round $R_{3,w}^*$.

rounds, and 3 rounds of $E_{3,w}$, respectively, which implies the maximum probabilities of differential trails are 2^{-7} , 2^{-42} and 2^{-91} , respectively. From the bound results of first three rounds, we also derive the lower bound $LB(r)$ of differential weight for any round r based on the following fact:

$$LB(r) = \max_{1 \leq a \leq r} (opt(a) + LB(r - a)), \quad LB(c) = opt(c) \text{ (for } c \text{ proven)}$$

where $opt(c)$ is the proved lower bound of c rounds, and $opt(r) \geq LB(r)$ always holds. We summarize the results in Table 7.

From the differential bound results of ChaCha Bahru, we therefore present differential trail bounds of different QSH variants in Table 8.

Table 8: Lower bound on probability of differential (pattern) trail, where $|\cdot|$ denotes the bit width of message or block size of permutation, respectively.

Variant	Rounds n_r+1	Ratio ($ message / permutation $)	Trail bound
QSH-512	10	512/1024	2^{-274}
QSH-768	19	1024/2048	2^{-547}
QSH-1024	19	1024/2048	2^{-547}

For QSH-512 the bound 2^{-274} is far below the collision claim of 2^{-256} ; for QSH-768 and QSH-1024 the bound 2^{-547} likewise dominates the collision claims of 2^{-384} and 2^{-512} . Single-trail differential distinguishers are therefore excluded with a wide margin.

7.3.2 Linear-trail bound property

The linear-cryptanalysis propagation rule through G is identical to the differential rule: any of four input masks of G are non-zero implies that there exist at least one non-zero output linear mask, hence the G call is counted as active. Similar to the propagation used in the differential analysis, we track the minimum weight of active G in any linear pattern trail through $E_{3,w}$ at the word level. Also, this rule is an over-approximation of bit-level activity, so the counts below are LOWER BOUNDS at the bit level.

Per- G linear bound. The linear cryptanalysis of a function F uses the *bias*

$$\epsilon_F(\alpha, \beta) = \Pr_x[\langle \alpha, x \rangle = \langle \beta, F(x) \rangle] - \frac{1}{2}, \in [-\frac{1}{2}, \frac{1}{2}],$$

where α, β are masks of input and output of F , the operation $\langle \cdot, \cdot \rangle$ denotes inner product. Or, equivalently we can use the *correlation* $c_F(\alpha, \beta) = 2\epsilon_F(\alpha, \beta) \in [-1, 1]$. Trail correlations multiply by Matsui's piling-up lemma: $|c|_{\text{trail}} = \prod_i |c|_i$ [39]. Also, Wallén's analysis of linear approximations of modular addition [48] gives, for every non-trivial mask triple (α, β, γ) of $c = a + b \bmod 2^w$, $|c|_{\text{add}}^{\max} \leq 2^{-1}$.

Similar to linear approximation table (LAT) of an S-box [39], we also introduce *linear pattern approximation table* (LPAT) of quadruple function $G : \mathbb{F}_{2^w}^4 \rightarrow \mathbb{F}_{2^w}^4$ to characterize its propagation of linear mask patterns over words.

Definition 2 (Linear Pattern Approximation Table). Let $e = (e_0, e_1, e_2, e_3)$, $e' = (e'_0, e'_1, e'_2, e'_3) \in \mathbb{F}_2^4$ be input mask pattern and output mask pattern of quadruple transformation $G : \mathbb{F}_{2^w}^4 \rightarrow \mathbb{F}_{2^w}^4$, respectively. The entry $lw(e, e')$ of function G 's LAPT is defined as $lw(e, e') = \log_2(lc(e, e'))$, i.e. weight of linear approximation's correlation where $lc(e, e')$ is:

$$\max_{\substack{\text{nz}(\Gamma_i)=e_i, \text{nz}(\Gamma'_i)=e'_i, \\ \text{for } i=0,1,2,3}} \left(\frac{2 \cdot |\{x = (x_0, x_1, x_2, x_3) \in \mathbb{F}_{2^w}^4 \text{ s.t. } \langle \Gamma, x \rangle = \langle \Gamma', G(x) \rangle\}|}{2^{4w}} - 1 \right)$$

where $\text{nz} : \mathbb{F}_{2^w} \rightarrow \mathbb{F}_2$ is non-zero indicator function, that is, $\text{nz}(x) = 1$, if $x \neq 0$, $\text{nz}(x) = 0$ if $x = 0$, and $\Gamma = (\Gamma_0, \Gamma_1, \Gamma_2, \Gamma_3)$ and $\Delta' = (\Gamma'_0, \Gamma'_1, \Gamma'_2, \Gamma'_3)$.

Based on the SAT method [38], we built the SAT models to search linear trail for G , and computed the LPAT of G given in Table 9. Furthermore, we also built SAT models of $E_{3,w}$ to search linear trails, and got the optimal weight 7 for 1 round, and lower bound 20 up to 2 rounds. That is, the maximum correlation of linear trails are 2^{-7} for 1 round, and 2^{-20} for 2 rounds.

Table 9: Linear pattern approximation table (LPAT) of G when $w = 32$. ‘-’ means correlation 0 for that pattern pair.

(e, e')	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0001	-	-	-	-	-	-	-	1	-	-	-	-	-	-	-	0
0010	-	-	-	-	-	-	-	-	-	-	31	0	-	-	-	-
0011	-	-	-	-	34	17	17	2	-	-	-	-	33	0	1	1
0100	-	-	-	-	-	7	-	0	-	-	-	-	-	-	-	-
0101	-	-	14	3	10	4	2	1	-	-	14	1	10	1	0	1
0110	-	-	-	-	-	-	-	-	-	-	-	-	13	1	1	0
0111	-	10	16	4	10	6	3	2	14	1	3	1	3	2	1	0
1000	-	-	-	-	-	-	-	-	-	-	-	16	-	15	-	1
1001	-	-	31	12	29	16	18	3	-	-	16	2	16	2	2	1
1010	-	24	37	22	34	21	19	3	-	16	19	16	17	3	2	1
1011	-	5	11	5	11	5	1	2	12	2	3	2	3	2	1	1
1100	-	-	-	-	-	-	-	-	-	-	-	1	-	2	0	0
1101	-	-	15	2	9	4	2	1	-	0	2	1	2	1	0	1
1110	-	20	29	3	11	5	2	1	-	1	2	1	3	0	1	0
1111	-	3	5	1	5	2	1	1	2	1	1	0	1	1	1	0

7.3.3 Rotational and Rotational-XOR property

Rotational cryptanalysis [33, 34, 35, 36] is a probabilistic chosen-plaintext attack exploiting the propagation of pairs of plaintexts $(x, x \lll r)$. For ARX-based schemes, only modular addition has rotational probability $p < 1$, which makes this attack effective. Thus the rotational probability of the whole ARX primitive can be estimated as that of all chained modular additions.

As an extension and refinement, Rotational-XOR (RX for short) property [3] tracks a pair $(x, \hat{x})$ whose words are related by a fixed left rotation of offset r and a given δ ,

$$\hat{x} = (x \lll r) \oplus \delta.$$

And the rotational case is $\delta = 0$. Also for ARX constructions, the only operation that does not commute with $(\cdot \lll r) \oplus \delta$ is modular addition. Consequently the cost of an Rotational-XOR characteristic is concentrated entirely in the modular additions it activates, and the propagation probability of it is denoted as DP_{RX} , the weight is $\omega = -\log_2 \text{DP}_{\text{RX}}$.

We assess CHACHA BAHRU at four levels, the quadruple map G , one round $R_{3,w}$, the permutation $E_{3,w}$, and the compression function $F_{3,w}$, and we are explicit throughout about which statements are *proven* and which rest on the standard (Markov) independence heuristic. Because $E_{3,w}$ uses no round constants, the analysis is invariant under the rotation direction; we fix left rotations and let r range over $\{1, \dots, w-1\}$, so the right-rotational cases appear as the $w-r$ cases.

Rotational and RX property for a single modular addition. For $z = x \boxplus y \bmod 2^w$ and a nonzero rotation offset r , Daum's analysis [22] gives the probability that the rotational relation is preserved,

$$p_+(r) = \frac{1}{4}(1 + 2^{r-w} + 2^{-r} + 2^{-w}), \quad \omega_+(r) = -\log_2 p_+(r). \quad (1)$$

Since $p_+(r) = p_+(w-r)$ the maximum is at $r = 1$, where for every $w \geq 16$

$$\omega_+(1) = -\log_2 \frac{3}{8} = 1.415 \text{ bits.}$$

And the following bound is also for one addition.

Lemma 1. For a single modular addition with independent uniform inputs and any offset r , the RX-differential probability is maximised by the trivial RX-difference ($\delta = 0$ on inputs and output) and equals $p_+(r)$. Hence every *isolated* active addition has weight at least $\omega_+(r) \geq \omega_+(1) = 1.415$ at the optimal offset.

The proof is the computation of [3] specialized to the all-MSB pattern; equivalently it follows from (1). We have to stress that Lemma 1 only bounds one addition in isolation and does *not* lower-bound the weight of a chained modular addition.

Chained Modular additions. As stated in [34], modular additions in a chain are *not* independent, *Chained modular additions do NOT form a Markov chain with respect to rotational differences. Rotational probabilities of chained modular additions cannot be computed as product of probabilities of the individual modular additions.* So the evaluation of the rotational property of CHACHA BAHRUhas to consider this independence. And the exact probability is given by the following result.

Lemma 2 (Khovratovich et al. [34], Lemma 2). Let $a_1, \dots, a_{k+1}$ be independent uniform w -bit words and let $0 < r < w$. The probability that every prefix sum preserves the rotational relation with offset r , i.e. $\Pr[(a_1 + a_2) \lll r = (a_1 \lll r) + (a_2 \lll r) \wedge \dots \wedge (a_1 + \dots + a_{k+1}) \lll r = (a_1 \lll r) + \dots + (a_{k+1} \lll r)]$, equals

$$P(k, w, r) = \frac{1}{2^{w(k+1)}} \binom{2^r + k}{k+1} \binom{2^{w-r} + k}{k+1}, \quad (2)$$

where k denotes the number of additions (equivalently, $k+1$ summands).

The quadruple function G and chained G^k . For a single G operation, the optimal RX trail has a weight of $5.660 = 4\omega_+(1)$, for both widths:

$$w=32 : (0, 80000000, 0, 1) \rightarrow (0, 40, 80000000, 1000000), \quad \omega = 5.660,$$

$$w=64 : (1, 0, 0, 0) \rightarrow (0, 8000, 1, 0), \quad \omega = 5.660.$$

And the optimal weight of rotational ($\delta = 0$) trail of G is $\omega = 2\omega(2) = 7.170$. Using the automatic search tools CASCADA [46], the RX optimum found by the tool is $k \times 5.660$ with input difference a single bit in the d -word collapsing to a rotational pair, while the $\delta = 0$ weight grows faster than $k \times 7.17$ because chaining couples the additions [33].

Table 10: The optimal weights of $\delta = 0$ and RX characteristic based on the search of CASCADA for chained G^k ($w = 32$, $r = 1$).

k	adds	$\delta = 0$ weight	Experimental	RX weight	Experimental
1	4	7.17	7.17	5.660	5.17
2	8	18.64	18.65	11.320	16.04
3	12	32.60	32.70	16.980	≥ 26.8
4	16	48.29	—	33.961	—
7	28	102.50	—	39.621	—
8	32	122.34	—	45.281	—
9	36	—	—	50.941	—
10	40	—	—	56.602	—
11	44	—	—	62.262	—
12	48	—	—	67.922	—

From one pass to round-reduce versions of $E_{3,w}$. A round $R_{3,w}$ is three G -layers (one column + two diagonal passes), 48 active G and 192 additions. Table 11 lists the optimal bound as the round of the permutation grows. As analyzed under the Markov assumption and the search of CASCADA, the collision- and preimage-resistance can be fulfilled by the deployed 10 (QSH-512) and 19 (QSH-768/1024) full rounds for rotation-tyle properties.

Table 11: Optimal RX trail of $E_{3,32}$ ($r = 1$).

target	fraction	adds	RX weight	time
first pass	1/3 round	64	$2^{-90.562}$	7.8 s
first+second pass	2/3 round	128	$2^{-181.125}$	44.0 s
one round	1 round	192	$2^{-271.687}$	70.2 s
two rounds	2 rounds	384	$2^{-543.374}$	251.7 s
three rounds	3 rounds	576	-	timeout

As a conclusion, CHACHA BAHRU and hence QSH, can be well-approximated secure against rotational and Rotational-XOR cryptanalysis, since enough security margins are maintained.

7.4 Analysis of the Algorithm QSH

7.4.1 Preimage attacks

In [14], Bhattacharyya et al. gave a preimage attack against the iterated JH compression function under the assumption that the underlying permutation is ideal. The attack targets an internal chaining value and proceeds by a two-sided multi-collision plus Meet-in-the-Middle (MitM): one builds a list of r backward values sharing the last half of a chaining state and a list of r forward values sharing the first half, and then matches the full $2b$ -bit state in the middle, read-

ing off the connecting message words afterwards. With $Q(r) \sim 2^{b(r-1)/r}(r!)^{1/r}$ the cost of an r -collision, a match is expected once $r \cdot Q(r) = 2^b$; for $b = 512$ the optimum is $r \approx 46$, giving $\approx 2^{507}$ queries, a marginal constant-factor gain over the 2^b generic bound. QSH exposes only the upper half ($m/2 = 512$ or 1024 bits) of an $h = 2048/4096$ -bit state. The internal multi-collision must therefore be built on a half-state of 1024 or 2048 bits, so the economics $r \cdot Q(r) = 2^{h/2}$ run on $h/2 \in \{1024, 2048\}$, i.e. the work is $\gtrsim 2^{1024}$. This is far above every classical claim of Table 5 and offers no shortcut. Also, the modular-addition injection and the ROOT flag break the clean XOR read-off of the connecting message words and constrain the backward direction (the lower half carries no post-permutation message addition), further reducing the usable freedom.

Automated MitM search. The MitM framework has been successfully applied to analyze preimage attacks on Merkle–Damgård with Davies–Meyer structure and sponge constructions in [2, 29] and [45, 23], respectively. Following the MitM framework [45, 23], we evaluate the resistance of QSH against preimage attacks. Different from the sponge structure, in QSH (JH-style), the match point will always involve the message at the upper part (hash output).

We implemented the framework in MILP language for QSH, and the unit level is set as word. As a result, for the word model, we find a solution with advantage of 8 word ($8 \times w$ bits) for 1 round of QSH, and collapse to 0 by the second full round.

7.4.2 Collision attacks

Following the collision attack framework [23], we also evaluate the resistance of QSH against collision attacks. The framework casts a collision as a meet-in-the-middle search directly for hash target or an inner (capacity) collision. The only change is the target, which is now a full match on the lower-half capacity ($m/2 = 32$ words) fed forward in the JH-style mode, with the 32 message words as the neutral source. The result mirrors the preimage case: a advantage exists for a single round (11 words) and collapses to 0 for two rounds.

7.4.3 Analysis in quantum setting

We now assess QSH in the quantum setting, refining the generic claims of Table 5. We work in the standard query model and distinguish, where it matters, the optimistic model that grants qubit-addressable quantum memory (qRAM) from the conservative model that does not.

Quantum (second-)preimage. Generically, a (second-)preimage of an n -bit digest is found by Grover search [26] in $\tilde{O}(2^{n/2})$ evaluations using only $O(n)$ qubits, which is the source of the $n/2$ -bit quantum (second-)preimage claim in Table 5. The question is whether a dedicated quantum MitM attack can do better. Quantum MitM techniques [31, 16, 47] replace the matching phase of a classical MitM by Grover/claw-finding, turning a balanced classical attack of

cost 2^{a+b} (with forward/backward chunks of cost $2^a, 2^b$) into roughly $2^{(a+b)/2}$; this beats Grover preimage only when the target admits an MitM decomposition whose chunks are substantially shorter than the full n -bit condition. QSH denies exactly this structure. Its wide pipe exposes only the truncated upper half (Section 5), so the n -bit preimage condition cannot be split into two independently computable chunks across the truncation boundary; and the word-level MitM search of Section 7.4.1 found no chunk separation surviving beyond two rounds of $E_{3,w}$ even classically. Since the quantum MitM speed-up only halves the exponent of an existing classical MitM separation, the absence of such a separation means no quantum MitM preimage on the full $E_{3,w}$ improves on generic Grover search. We therefore retain the $n/2$ -bit quantum (second-)preimage claim, with the deployed 10/19 rounds providing a large margin over the ≤ 2 -round reach of the MitM structure.

Quantum collision: generic bounds. For collisions, the generic picture is more nuanced than a single figure. In the optimistic qRAM model, the BHT algorithm [17] finds a collision in $\tilde{O}(2^{n/3})$ time but needs $\approx 2^{n/3}$ *quantum* memory; the $\Omega(2^{n/3})$ query lower bound of Aaronson–Shi [1], extended to random functions in the QROM by Zhandry [51], shows this is query-optimal. When qRAM is unavailable, the best known algorithm is Chailloux–Naya-Plasencia–Schrottenloher [18], which runs in $\tilde{O}(2^{2n/5})$ time with only $O(n)$ qubits and $2^{n/5}$ classical memory. Under realistic cost metrics that price memory access and circuit depth [11, 32], even the $2^{n/3}$ figure is not achievable cheaply, and classical parallel collision search remains competitive. Our claim of $n/3$ -bit quantum collision security (Table 5) is thus the most conservative reading; the realistic security level is plausibly closer to $2n/5$ or higher. As with the preimage case, the wide internal state gives the generic adversary no leverage: seeing only the n -bit truncated output, it faces a random function on n bits, so a quantum speed-up from the wide pipe would require an internal structural distinguisher, which we exclude in the above Sections.

7.5 Analysis of Specific Attacks

Length extension. The design of QSH adopts a wide-pipe strategy. The internal state is 2048/4096 bits while only 512/768/1024 bits are exposed as the final digest, providing generic attacks such as length-extension and certain second-preimage attacks. Also, length-extension attacks against QSH are prevented by the ROOT flag. The final compression call at the root always carries ROOT in $S_0^{(i-1)}$, while every intermediate call does not; an adversary who has observed a digest cannot forge a continuation because the ROOT-tagged state is unreachable from any non-root chain.

Multi-collisions. Joux-style multi-collisions for an n -bit hash require $t \cdot 2^{c/2}$ work for a 2^t -collision. With $c = m/2 \geq n$ across all three QSH variants, this is at least $t \cdot 2^{n/2}$, in line with the claimed collision resistance.

Free-start, semi-free-start, partial-bit attacks. Free-start collisions on $F_{d,w}$ (where the adversary chooses both $H^{(i-1)}$ and $M^{(i)}$ subject to a target output) cost $2^{h/2}$ on the random-permutation model, far above the claimed $2^{n/2}$. Partial-bit collisions on a k -bit subset of the digest are bounded by $\max(2^{k/2}, 2^{n/2}/\sqrt{n/k})$; the latter exceeds $2^{k/2}$ for any subset of size $k \leq n/2$, so the construction does not weaken under partial-bit attacks.

Statistical randomness (NIST SP 800-22). For each variant we generated a 10^9 -bit stream by hashing 8-byte little-endian counter values $0, 1, \dots, N-1$ and concatenating the digests, and we ran the *full* NIST SP 800-22 test suite—all 15 tests, including the frequency, block-frequency, runs, longest-run, binary-matrix-rank, spectral (DFT), (non-)overlapping template-matching, Maurer’s universal, linear-complexity, serial, approximate-entropy, cumulative-sums and random-excursion(-variant) tests—with the official `sts-2.1.2` tool on $m = 1000$ sequences of $n = 10^6$ bits each at significance level $\alpha = 0.01$. Over the $3 \times 188 = 564$ resulting test instances, every uniformity-of- p -values statistic exceeds the 0.0001 threshold (the smallest observed value being 0.0024), and the proportion of passing sequences meets the acceptance bound ($\geq 980/1000$) for all instances except two marginal non-overlapping-template instances (978/1000 for QSH-512 and 979/1000 for QSH-1024). These two exceptions lie within the number of rejections expected by chance at the 3σ level, exhibit no systematic pattern, and are accompanied by acceptable uniformity p -values; QSH-768 passes all 188 instances with no exception. We therefore conclude that all three QSH variants pass the NIST SP 800-22 statistical test suite. The full reports are provided as `finalAnalysisReport_QSH-*.txt` in the 6th folder of the submission package.

8 Performance of QSH

This section reports the throughput measured for our software implementations.

8.1 Software Implementation

We provide three software implementations, summarized in Table 12. We have conducted the measure experiments for the three software implementation, and the results of throughput along with cycle/byte are listed in Table 13 and Table 14. All implementations were verified for functional correctness against the known-answer test (KAT) vectors of every variant, and the optimized and additional ones are bit-identical to the reference implementation for all instances and message lengths.

x86-64 AVX2 implementation. Table 13 reports throughput for all three QSH variants on the baseline x86-64 test platform: an Intel Xeon processor at 2.80 GHz running Debian GNU/Linux 13 (Linux 6.12, single core), compiled with GCC 14.2.0 at `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99`. The time-stamp-counter (TSC) rate was

Table 12: The three software implementations. The optimized build targets x86-64 (AVX2), and the additional build is the AArch64 NEON port.

Implementation	Source / target	Optimisation layer added
Reference	ISO C99, no platform-specific instructions	— (baseline)
Optimised	ISO C99 with AVX2 explicit SIMD (<code>#if defined(_AVX2_)</code>); scalar fallback otherwise	within-permutation SIMD: the 16 mutually independent G of each permutation layer are evaluated across the AVX2 lanes, with the full 64-word permutation state held in registers
Additional	ISO C99 with AArch64 NEON SIMD; scalar fallback otherwise	NEON within-permutation SIMD for $w = 32$ (QSH-512); 2-way chunk batching for $w = 64$ (QSH-768/QSH-1024)

Table 13: Throughput (MB/s) and cycles per byte (cpb) for all QSH variants on the x86-64 AVX2 test platform, across message sizes. TSC-calibrated frequency: 2.799 GHz; Each is the mean over repeated iterations (≥ 100 iterations at the 1 MiB window);

Msg size	QSH-512		QSH-768		QSH-1024	
	MB/s	cpb	MB/s	cpb	MB/s	cpb
1 KiB	271.5	10.31	160.1	17.49	159.7	17.53
4 KiB	308.7	9.07	204.5	13.69	205.3	13.64
64 KiB	328.1	8.53	229.4	12.21	229.4	12.21
1 MiB	328.7	8.52	231.2	12.11	230.9	12.13

calibrated to 2.80 GHz; all cpb measurements are expressed in TSC cycles per byte, which reflects wall-clock efficiency independently of core turbo frequency. The optimized build uses *within-permutation* SIMD: the 16 mutually independent G functions of each permutation layer are evaluated together across the AVX2 lanes, keeping the full 64-word permutation state in registers.

For short messages of 1–4 KiB the fixed per-call overhead dominates, yielding 160–309 MB/s (9.1–17.5 cpb). As the message size grows the per-call overhead is amortized and throughput rises smoothly, becoming essentially saturated by 4–64 KiB. From 64 KiB upward, QSH-512 stabilizes at about **328 MB/s (8.53 cpb)**, while QSH-768/QSH-1024 stabilise at about **229–231 MB/s (12.1–12.2 cpb)**. Across all message sizes QSH-512 ($w = 32$) is the fastest variant: at $w = 64$ the permutation performs twice as many rounds ($n_r + 1 = 3w/8$ per call), raising the per-byte cost of QSH-768/QSH-1024. This message-size sensitivity is a well-known feature of tree-hash constructions and is not specific to QSH.

ARM AArch64 NEON implementation. Table 14 reports the AArch64 results on the baseline ARM platform: Debian GNU/Linux 12 with GCC 12.2.0 (single core), self-calibrated frequency ≈ 3.39 GHz. The NEON port uses within-permutation SIMD for QSH-512 ($w = 32$) and 2-way chunk batching for QSH-768/QSH-1024 ($w = 64$). At the 1 MiB window the NEON path brings all three variants to about 22.8–23.4 cpb (144–148 MB/s), an $\approx 11.6\times$ improvement over the portable reference, which runs at ≈ 266 –271 cpb on this core. The gain exceeds the SIMD lane count because the reference is an optimized, fully portable scalar baseline; after vectorization the three variants converge to a similar cpb, with QSH-768/QSH-1024 marginally faster than QSH-512 at this operating point.

Table 14: Throughput (MB/s) and cycles/byte (cpb) at the self-calibrated CPU frequency ≈ 3.39 GHz, AArch64 test platform, 1 MiB messages (mean over ≥ 100 iterations).

Variant	Reference		Additional (NEON)	
	MB/s	cpb	MB/s	cpb
QSH-512	12.5	270.9	144.7	23.4
QSH-768	12.7	266.6	148.4	22.8
QSH-1024	12.7	266.3	148.3	22.9

Resource consumption. Working memory of the compression function is dominated by the state S of $h = 2048$ bits (QSH-512) or 4096 bits (QSH-768/1024) plus a single message block M of $h/2$ bits. Total transient state is therefore ≤ 1 KiB for QSH-512 and ≤ 2 KiB for QSH-768/1024, which fits comfortably in L1 cache. The chaining-value stack used by the tree mode holds at most $\lceil \log_2 \Pi \rceil + 1$ chaining values during processing, each $m/2$ bits wide; for the maximum message length $\ell < 2^{64}$ bits this bound is at most 54 entries of $m/2$ bits, i.e., ≤ 3 KiB for QSH-512 and ≤ 7 KiB for QSH-768/1024. These analytic bounds are consistent with the measured static code footprint of the optimised build (text segment ≈ 15.5 KiB).

References

- [1] Scott Aaronson and Yaoyun Shi. Quantum lower bounds for the collision and the element distinctness problems. *Journal of the ACM*, 51(4):595–605, 2004.
- [2] Kazumaro Aoki, Jian Guo, Krystian Matusiewicz, Yu Sasaki, and Lei Wang. Preimages for step-reduced SHA-2. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 578–597. Springer, 2009.
- [3] Tomer Ashur and Yunwen Liu. Rotational cryptanalysis in the presence of constants. *IACR Trans. Symmetric Cryptol.*, 2016(1):57–70, 2016.
- [4] Jean-Philippe Aumasson, Luca Henzen, Willi Meier, and Raphael C.-W. Phan. Blake: Sha-3 proposal. Submission to the NIST SHA-3 Competition, 2010. Version 1.3.
- [5] Jean-Philippe Aumasson, Willi Meier, Raphael C-W Phan, and Luca Henzen. The hash function blake. 2014.
- [6] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The simon and speck lightweight block ciphers. In *Proceedings of the 52nd annual design automation conference*, pages 1–6, 2015.
- [7] Christof Beierle, Alex Biryukov, Luan Cardoso dos Santos, Johann Großschädl, Léo Perrin, Aleksei Udovenko, Vesselin Velichkov, and Qingju Wang. Alzette: A 64-bit arx-box - (feat. CRAX and TRAX). In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 419–448. Springer, 2020.
- [8] Christof Beierle, Phil Hebborn, Gregor Leander, and Yevhen Perekuda. Integral resistance of block ciphers with key whitening by modular addition. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V*, *Lecture Notes in Computer Science*, pages 497–529. Springer, 2025.
- [9] Daniel J. Bernstein. ChaCha, a variant of Salsa20. <https://cr.yp.to/chacha/chacha-20080128.pdf>, 2008. Document date: 2008-01-28.
- [10] Daniel J Bernstein. The salsa20 family of stream ciphers. In *New stream cipher designs: the eSTREAM finalists*, pages 84–97. Springer, 2008.

- [11] Daniel J. Bernstein. Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? In *SHARCS '09 Workshop Record*, 2009.
- [12] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. ECRYPT Hash Workshop, 2007. <http://sponge.noekeon.org/SpongeFunctions.pdf>.
- [13] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [14] Rishiraj Bhattacharyya, Avradip Mandal, and Mridul Nandi. Security analysis of the mode of JH hash function. In Seokhie Hong and Tetsu Iwata, editors, *Fast Software Encryption, 17th International Workshop, FSE 2010, Seoul, Korea, February 7-10, 2010, Revised Selected Papers*, volume 6147 of *Lecture Notes in Computer Science*, pages 168–191. Springer, 2010.
- [15] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *J. Cryptol.*, 4(1):3–72, 1991.
- [16] Xavier Bonnetain, María Naya-Plasencia, and André Schrottenloher. Quantum security analysis of AES. *IACR Trans. Symmetric Cryptol.*, 2019(2):55–93, 2019.
- [17] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In Cláudio L. Lucchesi and Arnaldo V. Moura, editors, *LATIN '98: Theoretical Informatics*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.
- [18] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 211–240. Springer, 2017.
- [19] Joan Daemen, Bart Mennink, and Gilles Van Assche. Sound hashing modes of arbitrary functions, permutations, and block ciphers. *IACR Trans. Symmetric Cryptol.*, 2018(4):197–228, 2018.
- [20] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.
- [21] Ivan Damgård. A design principle for hash functions. In Gilles Brassard, editor, *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 416–427. Springer, 1989.

- [22] Magnus Daum. *Cryptanalysis of Hash functions of the MD4-family*. PhD thesis, Bochum, Univ., Diss., 2005, 2005.
- [23] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, Lecture Notes in Computer Science, pages 3–37. Springer, 2024.
- [24] Niels Ferguson, Stefan Lucks, Bruce Schneier, Doug Whiting, Mihir Bellare, Tadayoshi Kohno, Jon Callas, and Jesse Walker. The skein hash function family. *Submission to NIST (round 3)*, 7(7.5):3, 2010.
- [25] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. Milp-based automatic search algorithms for differential and linear trails for speck. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, Lecture Notes in Computer Science, pages 268–288. Springer, 2016.
- [26] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*, pages 212–219. ACM, 1996.
- [27] Aldo Gunsing. Block-cipher-based tree hashing. Cryptology ePrint Archive, Paper 2022/283, 2022.
- [28] Aldo Gunsing, Joan Daemen, and Bart Mennink. Errata to sound hashing modes of arbitrary functions, permutations, and block ciphers. *IACR Trans. Symmetric Cryptol.*, 2020(3):362–366, 2020.
- [29] Jian Guo, Haoran Li, Meicheng Liu, Shichang Wang, and Tianyu Zhang. Dual-syncopation meet-in-the-middle attacks: New results on SHA-2 and MD5. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology - EUROCRYPT 2026 - 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Rome, Italy, May 10-14, 2026, Proceedings, Part VI*, Lecture Notes in Computer Science, pages 121–151. Springer, 2026.
- [30] Deukjo Hong, Jung-Keun Lee, Dong-Chan Kim, Daesung Kwon, Kwon Ho Ryu, and Dong-Geon Lee. Lea: A 128-bit block cipher for fast encryption on common processors. In *international workshop on information security applications*, pages 3–27. Springer, 2013.
- [31] Akinori Hosoyamada and Yu Sasaki. Quantum demirci-selçuk meet-in-the-middle attacks: Applications to 6-round generic feistel constructions. In Dario Catalano and Roberto De Prisco, editors, *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy,*

- September 5-7, 2018, Proceedings*, volume 11035 of *Lecture Notes in Computer Science*, pages 386–403. Springer, 2018.
- [32] Samuel Jaques and John M. Schanck. Quantum cryptanalysis in the RAM model: Claw-finding attacks on SIKE. In *CRYPTO 2019*, volume 11692 of *LNCS*, pages 32–61, 2019.
 - [33] Dmitry Khovratovich and Ivica Nikolic. Rotational cryptanalysis of ARX. In Seokhie Hong and Tetsu Iwata, editors, *Fast Software Encryption, 17th International Workshop, FSE 2010, Seoul, Korea, February 7-10, 2010, Revised Selected Papers*, Lecture Notes in Computer Science, pages 333–346. Springer, 2010.
 - [34] Dmitry Khovratovich, Ivica Nikolic, Josef Pieprzyk, Przemyslaw Sokolowski, and Ron Steinfeld. Rotational cryptanalysis of ARX revisited. In Gregor Leander, editor, *Fast Software Encryption - 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers*, Lecture Notes in Computer Science, pages 519–536. Springer, 2015.
 - [35] Dmitry Khovratovich, Ivica Nikolic, and Christian Rechberger. Rotational rebound attacks on reduced skein. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings*, Lecture Notes in Computer Science, pages 1–19. Springer, 2010.
 - [36] Dmitry Khovratovich, Ivica Nikolic, and Christian Rechberger. Rotational rebound attacks on reduced skein. *J. Cryptol.*, 27(3):452–479, 2014.
 - [37] Helger Lipmaa and Shiho Moriai. Efficient algorithms for computing differential properties of addition. In *Fast Software Encryption (FSE) 2001*, volume 2355 of *Lecture Notes in Computer Science*, pages 336–350. Springer, 2001.
 - [38] Yunwen Liu, Qingju Wang, and Vincent Rijmen. Automatic search of linear trails in ARX with applications to SPECK and chaskey. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve A. Schneider, editors, *Applied Cryptography and Network Security - 14th International Conference, ACNS 2016, Guildford, UK, June 19-22, 2016. Proceedings*, volume 9696 of *Lecture Notes in Computer Science*, pages 485–499. Springer, 2016.
 - [39] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1993.
 - [40] Ralph C. Merkle. One way hash functions and DES. In Gilles Brassard, editor, *Advances in Cryptology - CRYPTO '89, 9th Annual International*

- Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 428–446. Springer, 1989.
- [41] Ralph Charles Merkle. *Secrecy, authentication, and public key systems*. Stanford university, 1979.
 - [42] Dustin Moody, Souradyuti Paul, and Daniel Smith-Tone. Improved indistinguishability security bound for the JH mode. *Des. Codes Cryptogr.*, 79(2):237–259, 2016.
 - [43] National Institute of Standards and Technology. Secure hash standard (shs). Federal Information Processing Standards Publication 180-4, August 2015.
 - [44] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. Blake3: One function, fast everywhere. <https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf>, 2020. Specification and reference implementation.
 - [45] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 158–188. Springer, 2023.
 - [46] Adrián Ranea and Vincent Rijmen. Characteristic automated search of cryptographic algorithms for distinguishing attacks (CASCADA). *IET Inf. Secur.*, 16(6):470–481, 2022.
 - [47] André Schrottenloher and Marc Stevens. Simplified MITM modeling for permutations: New (quantum) attacks. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part III*, volume 13509 of *Lecture Notes in Computer Science*, pages 717–747. Springer, 2022.
 - [48] Johan Wallén. Linear approximations of addition modulo 2^n . In *Fast Software Encryption (FSE) 2003*, volume 2887 of *Lecture Notes in Computer Science*, pages 261–273. Springer, 2003.
 - [49] Hongjun Wu. The hash function jh, 2008. <https://personal.ntu.edu.sg/wuhj/research/jh/index.html>.
 - [50] Hongjun Wu. The hash function JH. Submission to the NIST SHA-3 competition (Round 3), 2011. https://www3.ntu.edu.sg/home/wuhj/research/jh/jh_round3.pdf.

- [51] Mark Zhandry. A note on the quantum collision and set equality problems.
Quantum Inf. Comput., 15(7&8):557–567, 2015.

Appendix

A Algorithm 1 when $d = 3$

Table 15, 16 and 17 spell out the three passes of $R_{3,w}$ at the level of individual word indices. Each row corresponds to one G call and lists the four words it consumes, named by the index $i = 16x_0 + 4x_2 + x_1$ for the cube ($d = 3$) ; the parameter column identifies the line. For the column pass (Table 15), sweeping x_0 at a fixed (x_1, x_2) yields the stride-16 quadruple $\{b, b+16, b+32, b+48\}$ with $b = x_1 + 4x_2$. The two coupling passes (Table 16 and 17) instead enumerate the diagonals of the (x_0, x_1) - and (x_1, x_2) -planes, indexed by the diagonal offset x'_1 (resp. x'_2) together with the slice coordinate x_2 (resp. x_0); these tables already absorb the P_{16}/P_{16}^{-1} re-indexing, so the index sets shown are the operands actually fed to G . In every pass the sixteen quadruples are pairwise disjoint and cover all 64 words , so each pass is a partition of the state: every word enters exactly one G per pass.

Table 15: CHACHA BAHRU round $R_{3,w}$, G along x_0 with index $i = 16x_0 + 4x_2 + x_1$, where param denotes the index parameters of step 8 in Algorithm 1.

param	words fed to G			
$(x_1, x_2)=(0, 0)$	A_0	A_{16}	A_{32}	A_{48}
$(x_1, x_2)=(1, 0)$	A_1	A_{17}	A_{33}	A_{49}
$(x_1, x_2)=(2, 0)$	A_2	A_{18}	A_{34}	A_{50}
$(x_1, x_2)=(3, 0)$	A_3	A_{19}	A_{35}	A_{51}
$(x_1, x_2)=(0, 1)$	A_4	A_{20}	A_{36}	A_{52}
$(x_1, x_2)=(1, 1)$	A_5	A_{21}	A_{37}	A_{53}
$(x_1, x_2)=(2, 1)$	A_6	A_{22}	A_{38}	A_{54}
$(x_1, x_2)=(3, 1)$	A_7	A_{23}	A_{39}	A_{55}
$(x_1, x_2)=(0, 2)$	A_8	A_{24}	A_{40}	A_{56}
$(x_1, x_2)=(1, 2)$	A_9	A_{25}	A_{41}	A_{57}
$(x_1, x_2)=(2, 2)$	A_{10}	A_{26}	A_{42}	A_{58}
$(x_1, x_2)=(3, 2)$	A_{11}	A_{27}	A_{43}	A_{59}
$(x_1, x_2)=(0, 3)$	A_{12}	A_{28}	A_{44}	A_{60}
$(x_1, x_2)=(1, 3)$	A_{13}	A_{29}	A_{45}	A_{61}
$(x_1, x_2)=(2, 3)$	A_{14}	A_{30}	A_{46}	A_{62}
$(x_1, x_2)=(3, 3)$	A_{15}	A_{31}	A_{47}	A_{63}

B Proof of Proposition 1

Throughout, $\mathbf{e} = 2^{w-1}$ is the most-significant bit (MSB).

Lemma 3. For $z = x \boxplus y \bmod 2^w$, a difference $(\alpha, \beta \rightarrow \gamma)$ has $\text{DP} = 1$ iff $\alpha, \beta, \gamma \in \{0, \mathbf{e}\}$ and $\gamma = \alpha \oplus \beta$.

Table 16: CHACHA BAHRU round $R_{3,w}$, diagonal coupling (x_0, x_1) within slice x_2 , where param denotes the index parameters of step 8 in Algorithm 1.

param	Quadruple words fed to G			
$(x'_1, x_2)=(0, 0)$	A_0	A_{17}	A_{34}	A_{51}
$(x'_1, x_2)=(1, 0)$	A_1	A_{18}	A_{35}	A_{48}
$(x'_1, x_2)=(2, 0)$	A_2	A_{19}	A_{32}	A_{49}
$(x'_1, x_2)=(3, 0)$	A_3	A_{16}	A_{33}	A_{50}
$(x'_1, x_2)=(0, 1)$	A_4	A_{21}	A_{38}	A_{55}
$(x'_1, x_2)=(1, 1)$	A_5	A_{22}	A_{39}	A_{52}
$(x'_1, x_2)=(2, 1)$	A_6	A_{23}	A_{36}	A_{53}
$(x'_1, x_2)=(3, 1)$	A_7	A_{20}	A_{37}	A_{54}
$(x'_1, x_2)=(0, 2)$	A_8	A_{25}	A_{42}	A_{59}
$(x'_1, x_2)=(1, 2)$	A_9	A_{26}	A_{43}	A_{56}
$(x'_1, x_2)=(2, 2)$	A_{10}	A_{27}	A_{40}	A_{57}
$(x'_1, x_2)=(3, 2)$	A_{11}	A_{24}	A_{41}	A_{58}
$(x'_1, x_2)=(0, 3)$	A_{12}	A_{29}	A_{46}	A_{63}
$(x'_1, x_2)=(1, 3)$	A_{13}	A_{30}	A_{47}	A_{60}
$(x'_1, x_2)=(2, 3)$	A_{14}	A_{31}	A_{44}	A_{61}
$(x'_1, x_2)=(3, 3)$	A_{15}	A_{28}	A_{45}	A_{62}

Table 17: CHACHA BAHRU round $R_{3,w}$, diagonal coupling (x_1, x_2) within slice x_0 , where param denotes the index parameters of step 8 in Algorithm 1.

param	Quadruple words fed to G			
$(x_0, x'_2)=(0, 0)$	A_0	A_5	A_{10}	A_{15}
$(x_0, x'_2)=(0, 1)$	A_4	A_9	A_{14}	A_3
$(x_0, x'_2)=(0, 2)$	A_8	A_{13}	A_2	A_7
$(x_0, x'_2)=(0, 3)$	A_{12}	A_1	A_6	A_{11}
$(x_0, x'_2)=(1, 0)$	A_{16}	A_{21}	A_{26}	A_{31}
$(x_0, x'_2)=(1, 1)$	A_{20}	A_{25}	A_{30}	A_{19}
$(x_0, x'_2)=(1, 2)$	A_{24}	A_{29}	A_{18}	A_{23}
$(x_0, x'_2)=(1, 3)$	A_{28}	A_{17}	A_{22}	A_{27}
$(x_0, x'_2)=(2, 0)$	A_{32}	A_{37}	A_{42}	A_{47}
$(x_0, x'_2)=(2, 1)$	A_{36}	A_{41}	A_{46}	A_{35}
$(x_0, x'_2)=(2, 2)$	A_{40}	A_{45}	A_{34}	A_{39}
$(x_0, x'_2)=(2, 3)$	A_{44}	A_{33}	A_{38}	A_{43}
$(x_0, x'_2)=(3, 0)$	A_{48}	A_{53}	A_{58}	A_{63}
$(x_0, x'_2)=(3, 1)$	A_{52}	A_{57}	A_{62}	A_{51}
$(x_0, x'_2)=(3, 2)$	A_{56}	A_{61}	A_{50}	A_{55}
$(x_0, x'_2)=(3, 3)$	A_{60}	A_{49}	A_{54}	A_{59}

Proof. By the Lipmaa–Moriai weight formula [37], $\text{wt} = 0$ forces $\alpha_i = \beta_i = \gamma_i$ for $0 \leq i \leq w - 2$; the validity condition $\alpha \oplus \beta \oplus \gamma \oplus (\beta \ll 1) = 0$ then forces the common low part to vanish, leaving $\alpha, \beta, \gamma \in \{0, \mathbf{e}\}$ with $\gamma = \alpha \oplus \beta$. The converse is immediate since $x \oplus \mathbf{e} = (x \boxplus \mathbf{e}) \bmod 2^w$. $\square$

(1) No non-trivial weight-0 entry

Proof of Proposition 1(1). Let the input difference be (A, B, C, D) and assume weight 0; then each addition is weight-0, so by Lemma 3 every addend and output lies in $\{0, \mathbf{e}\}$. Let A', C' be the differences of a, c after additions 1, 2 (G updates in place).

Add. 1 ($a \oplus b \lll 1$): $A \in \{0, \mathbf{e}\}$, $B \lll 1 \in \{0, \mathbf{e}\}$ so $B \in \{0, 2^{w-2}\}$, $A' = A \oplus (B \lll 1) \in \{0, \mathbf{e}\}$, and $D' = (D \oplus A') \lll \tau_1$.

Add. 2 ($c \oplus d' \lll \sigma_2$): $C \in \{0, \mathbf{e}\}$ and, as $\tau_1 + \sigma_2 = w - 1$, $(D \oplus A') \lll (w - 1) \in \{0, \mathbf{e}\} \iff D \oplus A' \in \{0, 1\}$; also $C' \in \{0, \mathbf{e}\}$, $B' = (B \oplus C') \lll \tau_2$.

Add. 3 ($a \oplus b', \sigma_3 = 0$): $B' \in \{0, \mathbf{e}\}$, i.e. $B \oplus C' \in \{0, 2^{w/2+3}\}$. Since $B \in \{0, 2^{w-2}\}$, $C' \in \{0, 2^{w-1}\}$ and $2^{w/2+3} \notin \{2^{w-2}, 2^{w-1}\}$ for $w \geq 16$, the two sets meet only in 0; hence $B = C' = 0$, so $A' = A$, $A'' = A$, $D'' = (D' \oplus A) \lll \tau_3$.

Add. 4 ($c \oplus d'' \lll \sigma_4$): with $\tau_3 + \sigma_4 = \frac{w}{2}$, $(D' \oplus A) \lll \frac{w}{2} \in \{0, \mathbf{e}\} \iff D' \oplus A \in \{0, 2^{w/2-1}\}$.

Finally $A \in \{0, \mathbf{e}\}$ and $D' = (D \oplus A) \lll \frac{w}{2}$ (as $\tau_1 = \frac{w}{2}$). If $D \oplus A = 0$ then $D' \oplus A = A$, and membership in $\{0, 2^{w/2-1}\}$ forces $A = 0$, hence $(A, B, C, D) = 0$. If $D \oplus A = 1$ then $D' \oplus A \in \{2^{w/2}, 2^{w/2} \oplus 2^{w-1}\}$, disjoint from $\{0, 2^{w/2-1}\}$ — a contradiction. Thus the only weight-0 difference is 0, and $\text{DP}_G^{\max} \leq 2^{-1}$. The argument uses only $\sigma_1 = 1$, $\tau_1 = \frac{w}{2}$, $\tau_1 + \sigma_2 \equiv w - 1$, $\tau_3 + \sigma_4 \equiv \frac{w}{2} \pmod{w}$ and $2^{(w-1)-\tau_2} \notin \{2^{w-2}, 2^{w-1}\}$, hence holds for every admissible width $w \geq 16$. $\square$

(2)–(3) Support, width-monotonicity, and branch number

A pair (e, e') lies in S_G iff there exist concrete differences with that activity pattern together with a differential characteristic of G whose four additions are each Lipmaa–Moriai-valid (rotations and XORs being deterministic). For each of the 256 pattern pairs this is a finite decision problem; solving it exhaustively yields the *same* relation S_G for $w \in \{16, 32, 64\}$, listed in Table 18, proving the support part of (2). The minimum-weight cell values wt_w were obtained by minimising the summed Lipmaa–Moriai weights subject to validity and the activity pattern. We find $\text{wt}_{32}(e, e') \geq \text{wt}_{16}(e, e')$ for all 135 non-trivial cells, and $\text{wt}_{64}(e, e') \geq \text{wt}_{32}(e, e')$ for all cells of weight at most 9; equality holds precisely on the low-weight cells, whose optimal characteristic is confined to the low-order bits and is therefore width-independent, while the higher-weight cells grow strictly with w . This establishes the width-monotonicity of (2). For (3), G is a bijection, so $S_G(0) = \{0\}$ and $e \neq 0 \Rightarrow e' \neq 0$, giving $|e| + |e'| \geq 2$; the value 2 would require $|e| = |e'| = 1$, but Table 18 contains no admissible weight-1/weight-1 pair, whereas $(1000) \mapsto (0011)$ attains $|e| + |e'| = 3$, so the branch number equals 3. $\square$

Table 18: The width-independent difference support S_G ($w \in \{16, 32, 64\}$): for each input pattern $e = (a\ b\ c\ d)$ the admissible output patterns $e' \in S_G(e)$; a bit is 1 iff that word is active.

e	$S_G(e)$
0000	0000
0001	1010, 1011, 1101, 1110, 1111
0010	1011, 1101, 1111
0011	0111, 1010, 1011, 1101, 1110, 1111
0100	0011, 0101, 0111, 1001, 1010, 1011, 1101, 1110, 1111
0101	0011, 0100, 0101, 0111, 1001, 1010, 1011, 1101, 1110, 1111
0110	0011, 0101, 0111, 1001, 1010, 1011, 1100, 1101, 1110, 1111
0111	0010, 0011, 0101, 0110, 0111, 1001, 1010, 1011, 1100, 1101, 1110, 1111
1000	0011, 0101, 0111, 1010, 1011, 1101, 1110, 1111
1001	0011, 0101, 0111, 1010, 1011, 1101, 1110, 1111
1010	0011, 0101, 0111, 1000, 1010, 1011, 1101, 1110, 1111
1011	0010, 0011, 0101, 0110, 0111, 1000, 1010, 1011, 1101, 1110, 1111
1100	0011, 0101, 0111, 1001, 1010, 1011, 1101, 1110, 1111
1101	0001, 0011, 0100, 0101, 0111, 1001, 1010, 1011, 1101, 1110, 1111
1110	0011, 0101, 0110, 0111, 1001, 1010, 1011, 1100, 1101, 1110, 1111
1111	0001, 0010, 0011, 0101, 0110, 0111, 1001, 1010, 1011, 1100, 1101, 1110, 1111

C Lower Bound on Active G

In Table 19, we list the lower bound on the minimum number of active G functions in any differential (pattern) trail through CHACHA BAHRU $E_{3,w}$.

Table 19: Lower bound $LB(r)$ on the minimum number of active G functions in any differential (pattern) trail through CHACHA BAHRU $E_{3,w}$. Rounds 1–6: obtained by direct optimization; larger rounds: bound via partition.

CHACHA BAHRU-32 ($n_r+1 = 10$)			CHACHA BAHRU-64 ($n_r+1 = 19$)		
Full rounds r	$LB(r)$	Partition	Full rounds r	$LB(r)$	Partition
1	4	1	1	4	1
2	13	2	2	13	2
3	21	3	3	21	3
4	31	4	4	31	4
5	39	5	5	39	5
6	49	6	6	49	6
7	53	$6 + 1$	7	53	$6 + 1$
8	62	$6 + 2$	8	62	$6 + 2$
9	70	$6 + 3$	9	70	$6 + 3$
10*	71	$6 + 3 + 1$	10	80	$6 + 4$
			11	88	$6 + 5$
			12	98	$6 + 6$
			13	102	$6 + 6 + 1$
			14	111	$6 + 6 + 2$
			15	119	$6 + 6 + 3$
			16	129	$6 + 6 + 4$
			17	137	$6 + 6 + 5$
			18	147	$6 + 6 + 6$
			19*	148	$6 + 6 + 6 + 1$

* Parameter round used by the corresponding CHACHA BAHRU instance.